



JOKU OTSIKKO

Omar Mayani

Pro gradu -tutkielma
Kesäkuu 2020

MATEMATIIKAN JA TILASTOTIETEEN LAITOS

Tarkastajat:

Prof. Ion Petre

Markus Ylikojola

Turun yliopiston laatu järjestelmän mukaisesti tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck-järjestelmällä

TURUN YLIOPISTO, Matematiikan ja tilastotieteen laitos

Pro gradu -tutkielma

Pääaine: Matematiikka

Tekijä: Omar Mayani

Otsikko: Joku otsikko

Ohjaaja: Prof. Ion Petre

Sivumäärä: xx sivua + liitteet xx sivua

Aika: Kesäkuu 2020

Kirjoita tähän tiivistelmä. Laita `\noindent`-komento kappaleen alkuun, niin $\text{\LaTeX}$ ei sisennä ensimmäistä riviä.

Komento `\vspace` taas jättää sopivan välin kappaleiden väliin - 4mm näyttää aika hyvältä.

Kirjoita tiivistelmä napakasti ja kaikenlaista toistoa välttäen.

Asiasanat: tiivistelmäsivu, Pro gradu -tutkielma, $\text{\LaTeX}$ -ladontajärjestelmä.



Teknologiateollisuus

Tämän pro gradu -tutkielman tutkimustyötä on tuettu Teknologiateollisuuden 100-vuotissäätiön myöntämällä apurahalla.

ON NOTATION AND TERMINOLOGY

Unless otherwise specified, we use the following notations and terminologies throughout the thesis.

$\mathbb{R}$	the set of real numbers,
$\mathcal{L}$	a loss function,
$\mathbf{T}$	the transpose of a matrix,
w	a member of a vocabulary (usually a word),
$V = \{w_1, w_2, \dots, w_{ V }\}$	a (finite) vocabulary,
$\mathbf{w}$	the vector representation of w ,

Sisällys

1	Johdanto	4
2	Diskreetit tekstiesitykset ja klassinen kielimallinnus	7
2.1	Tokenien one-hot-enkoodaus	7
2.2	N-gram-mallit	8
3	Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus	10
3.1	Upotusvektorit	10
3.2	Upotusvektorien oppiminen Word2Vec-menetelmällä	11
3.3	N-gram-malli optimointiongelmiana	13
3.4	Monikerroksinen perseptroni seuraavan tokenin ennustajana	15
4	Gradienttimenetelmä	18
4.1	Häiriöalttius ja optimoinnin haasteet	21
4.2	Adaptiiviset optimointimenetelmät	22
5	Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille	26
5.1	Itsehuomio	26
5.2	Kausaalinen itsehuomio	28
5.3	Monipäinen itsehuomio	29
5.4	Transformer-dekooderikerros	30
5.5	Seuraavan tokenin ennustaminen ja kielimallin tavoitefunktio	31
5.6	Autoregressiivinen generointi	32
5.7	Skaalaus ja jälkikoulutus	33
5.8	earlier subsection that should come later	34
5.9	Parametrisen kielimallin rajoitteet ja yhteys RAG-järjestelmiin	34
6	Metodologia	36
6.1	Tutkimusasetelma	36
6.2	Tutkimusaineisto	36
6.3	Cortex Analyst ja semanttinen näkymä	36
6.4	Arviointiperusteet	38
7	Kokeet	40
8	Tulokset	43
9	Rajoitukset	48
10	Johtopäätökset	49

1 Johdanto

Kielimalli on autoregressiivinen malli, joka ennustaa seuraavan sanan (tai tokenin) aiemman kontekstin perusteella [15, 2]. Kun mallin tuottama sana liitetään aiemman kontekstiin, kutsutaan mallia päivitetyllä kontekstilla, jolloin se voi tuottaa pitkiäkin tekstijaksoja. Suuret kielimallit ovat viime vuosina kehittyneet pelkistä tekstin tuottajista järjestelmiksi, jotka kykenevät myös hyödyntämään ulkoisia työkaluja [34, 46]. Jos kielimallia suoritettava ympäristö tulkitsee tiettyjä mallin tuottamia merkintöjä työkalukutsuina, kielimalli voi käyttää esimerkiksi laskentaa, verkkohakua tai tietokantakyselyitä.

Luonnollisen kielen käyttö tietokantojen rajapintana (Text-to-SQL) on kiinnostava sovelluskohde, koska se voi madaltaa datan hyödyntämisen kynnyistä [19, 9]. Tällöin käyttäjän ei tarvitse osata SQL-kieltä eikä tuntea tietokannan rakennetta yksityiskohtaisesti, vaan hän voi esittää tiedontarpeensa tavallisella kielellä. Käytännön hyöty riippuu kuitenkin siitä, kuinka luotettavasti järjestelmä kykenee tulkitsemaan käyttäjän tarkoituksen ja muuntamaan sen oikeaksi SQL-kyselyksi [47, 17].

Tässä tutkielmassa tarkastellaan Snowflaken Cortex Analyst -järjestelmää taustatutkimuksena [39]. Cortex Analyst on kielimallipohjainen työkalu, joka muodostaa käyttäjän luonnollisen kielen kysymyksistä SQL-kyselyitä ja voi suorittaa niitä tietokantaa vasten. Tutkimuskohteena on puhelinkeskusdataa sisältävä tietokanta, jossa analyysin keskiössä on yksi keskusteludataa sisältävä päätason taulu sekä kolme sitä täydentävää taulua (ks. kuva 1). Järjestelmän toiminta perustuu semanttiseen näkymään, jossa taulut, kentät ja niiden merkitykset kuvataan YAML-muodossa (ks. kuva 2) [38]. Tämä näkymä toimii linkkinä käyttäjän luonnollisen kielen kysymyksen ja tietokannan rakenteen välillä.

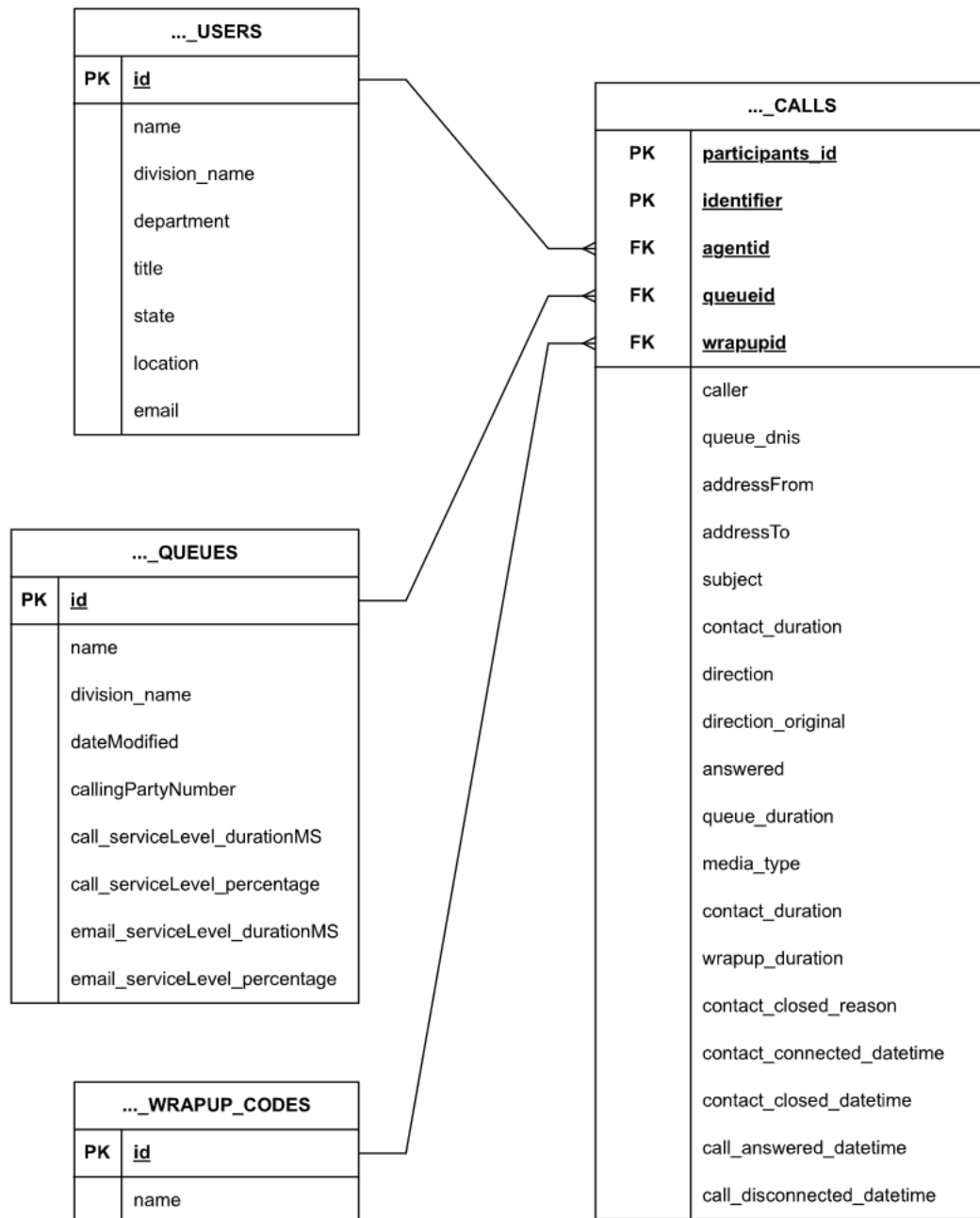
Tutkielman tavoitteena on arvioida, kuinka hyvin Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin ja millaisia rajoituksia sen käyttöön liittyy. Koska nykyaikaisten kaupallisten kielimallien tarkat toteutustiedot eivät yleensä ole julkisia, järjestelmän arviointi perustuu empiiriseen tarkasteluun [8]. Tässä työssä vertaillaan nimenomaan Cortex Analystin tuottamia SQL-kyselyitä eikä niiden palauttamia vastauksia [26]. Raja on perusteltu sekä aineiston luottamuksellisuuden vuoksi että siksi, että virheellisen kyselyn vaikutus lopulliseen vastaukseen riippuu myös tietokannan sisällöstä.

Tutkimusta ohjaavat seuraavat tutkimuskysymykset: Kuinka konsistentisti Cortex Analyst tuottaa SQL-kyselyitä, kun sama kysymys esitetään useita kertoja? Kuinka herkkä Cortex Analyst on sellaisille syötteen muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä? Millaisia virhe- ja vaihtelutyyppejä järjestelmän tuottamissa SQL-kyselyissä esiintyy?

Tutkielma etenee siten, että ensin kuvataan tutkimusasetelma, aineisto ja arviointitapa. Tämän jälkeen esitellään kokeet, joissa tarkastellaan järjestelmän konsistenssia ja herkkyyttä merkitykseltään vähäisille muutoksille. Lopuksi esitetään tulokset, johtopäätökset ja tutkimuksen keskeiset rajoitukset.

Attribute	Value
PARTICIPANTS_ID	ea0a918f
IDENTIFIER	63c44469
QUEUEID	3b10a230
CALLERS	+123456789012
AGENTID	b7e632de
SUBJECT	Null
ADDRESS_TO	Null
ADDRESS_FROM	Null
CONTACT_DURATION	363
DIRECTION	inbound
DIRECTION_ORIGINAL	inbound
ANSWERED	1
QUEUE_DURATION	2
CALL_DURATION	316
WRAPUPID	dd91b10d
WRAPUP_DURATION	47
CONTACT_CLOSED_REASON	client
CONTACT_CONNECTED_DATETIME	2025-08-23 13:41:13.000
CONTACT_CLOSED_DATETIME	2025-08-23 13:47:28.000
CALL_ANSWERED_DATETIME	2025-08-23 13:41:25.000
CALL_DISCONNECTED_DATETIME	2025-08-23 13:46:41.000
MEDIA_TYPE	call
CONTACT_CONNECTED_DATE	2025-08-23 00:00:00.000

Taulukko 1: Esimerkki yhdestä tietokannan rivistä. Osa kentistä on anonymisoitu tietosuoja- ja luottamuksellisuussyistä.



Kuva 1: Tutkimuksessa käytettävän tietokannan rakenne. Taulujen nimet on lyhennetty luettavuuden vuoksi.

2 Diskreetit tekstiesitykset ja klassinen kielimallinnus

Kielimallien yhteydessä tekstiä käsitellään tavallisesti diskreetteinä tokeneina, jotka muunnetaan laskennalliseen muotoon mallintamista varten. Olkoon $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto eli joukko kaikista mallin käyttämistä tokeneista ja olkoon $W = (w_1, w_2, \dots, w_{|W|})$ korpus eli havaittu tokenijono. Tällöin $w_t \in V$ merkitsee korpuksen ajanhetken t tokenia. Tokenilla voidaan tarkoittaa esimerkiksi sanaa, osasanaa, merkkiä tai muuta diskreettiä tekstiyksikköä riippuen käytetystä tokenisointimenetelmästä [15].

2.1 Tokenien one-hot-enkoodaus

Diskreetit tokenit esitetään numeerisessa muodossa, jotta niitä voidaan hyödyntää tilastollisissa malleissa ja neuroverkoissa. Yksinkertaisin ja yleisesti käytetty esitystapa on one-hot-enkoodaus. Siinä kukin token $v_j \in V$ esitetään vektorina $x(v_j) \in \{0, 1\}^{|V|}$, jossa

$$x(v_j)_k = \begin{cases} 1, & \text{jos } k = j, \\ 0, & \text{muulloin.} \end{cases}$$

Tällainen esitys ilmaisee ainoastaan tokenin identiteetin: eri tokeneilla on toisistaan poikkeavat vektoriesitykset, mutta niihin ei sisälly eksplisiittistä semanttista tai syntaktista rakennetta [21]. Menetelmän etuna on yksikäsitteisyys ja sen yksinkertainen yhteys sanaston indeksointiin.

Käytännön kielimallinnuksessa sanastoa täydennetään usein erikoistokeneilla, joilla kuvataan sekvenssin rakennetta ja käsitellään harvinaisia havaintoja. Tällaisia ovat esimerkiksi aloitus- ja lopetusmerkit $\langle s \rangle$ ja $\langle /s \rangle$ sekä tuntemattoman tokenin merkki $\langle unk \rangle$ [15].

Aloitus- ja lopetusmerkit ovat erityisen hyödyllisiä silloin, kun mallin ennustettavaa kontekstia halutaan määritellä myös sekvenssin reuna-alueilla. Esimerkiksi n -gram-malleissa (ks. osio 2.2) kontekstin pituus on vakio, minkä vuoksi sekvenssin alku- ja loppupäähän lisätään tyypillisesti riittävä määrä erikoistokeneita, jotta kontekstit voidaan muodostaa yhtenäisesti kaikille ajanhetkille.

Koko korpus voidaan esittää one-hot-vektoreiden jonona

$$X = (x_1, x_2, \dots, x_{|W|}),$$

missä $x_t = x(w_t)$ on tokenin w_t one-hot-esitys. Jos syöte koostuu useasta tokenista, voidaan niiden one-hot-esitykset yhdistää esimerkiksi ketjuttamalla ne yhdeksi syötevektoriksi

$$\tilde{x}_t = [x_{t-n+1}; x_{t-n+2}; \dots; x_{t-1}] \in \{0, 1\}^{(n-1)|V|}.$$

Tämä muodostaa luonnollisen syötteen monille kontekstipohjaisille malleille, sillä se säilyttää kontekstin tokenijärjestyksen eksplisiittisesti.

One-hot-enkoodaus muodostaa perustavanlaatuisen sillan tekstin ja numeeristen kielimallien välille. Se tarjoaa täsmällisen ja muodollisesti yksinkertaisen esitystavan, jonka varaan voidaan rakentaa sekä tilastollisia n -gram-malleja että myöhempiä neuroverkkopohjaisia kielimalleja.

2.2 N-gram-mallit

N-gram-malli on klassinen tilastollinen kielimalli, jossa sekvenssin seuraavan tokenin todennäköisyyttä estimoidaan rajallisen kontekstin perusteella [15]. Menetelmä perustuu $(n - 1)$:n kertaluvun Markov-oletukseen, jonka mukaan tokenin w_t todennäköisyys riippuu vain sitä välittömästi edeltävistä $(n - 1)$ tokenista eikä koko aiemmasta historiasta. Markov-oletuksen perusteella ehdollinen todennäköisyys voidaan approksimoida muodossa

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}),$$

missä w_t on ajanhetken t tokeni ja $(w_{t-n+1}, \dots, w_{t-1})$ muodostaa sitä edeltävän $(n - 1)$ tokenin kontekstin.

N-gram-mallien historialliset juuret yhdistetään usein Shannonin varhaiseen informaatioteoreettiseen työhön, jossa luonnollista kieltä tarkasteltiin symbolijonojen tilastollisena prosessina ja eri kertalukujen approksimaatioita havainnollistettiin englannin kielellä [35, 36].

Olkoon $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto, jossa $|V|$ on sanaston koko, ja $W = (w_1, w_2, \dots, w_{|W|})$ havaintoaineisto. N-gram-malli voidaan toteuttaa muodostamalla lukumatriisi $C \in \mathbb{R}^{|V|^{n-1} \times |V|}$, jossa $C_{c,j}$ edustaa sitä, kuinka monta kertaa kontekstia c seuraa tokeni v_j havaintoaineistossa. Formaalisti

$$C_{c,j} = |\{t \in \mathbb{N} \mid (w_{t-n+1}, \dots, w_{t-1}) = c \text{ ja } w_t = v_j\}|.$$

Kun lukumatriisin C rivit normalisoidaan jakamalla jokainen alkio rivinsä summalla, saadaan frekvenssimatriisi $F \in \mathbb{R}^{|V|^{n-1} \times |V|}$, jossa

$$F_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|V|} C_{c,k}},$$

ja joka edustaa ehdollista todennäköisyyttä

$$P(w_t = v_j \mid (w_{t-n+1}, \dots, w_{t-1}) = c).$$

Tekstin generointi n-gram-mallilla tapahtuu iteratiivisesti hyödyntämällä frekvenssimatriisia F . Ensin valitaan alkukonteksti, esimerkiksi erikoismerkeillä aloitettu $(n - 1)$ -tokenin pituinen alkujakso. Tämän jälkeen mallin tämänhetkistä kontekstia vastaava rivi F_c , tulkitaan todennäköisyysjakaumaksi, josta seuraava tokeni valitaan satunnaisotannalla. Kun tokeni w_t on generoitu, konteksti päivitetään siirtämällä ikkuna yhden askeleen eteenpäin siten, että uusi konteksti muodostuu edeltävistä $(n - 1)$ generoidusta tokenista. Prosessia jatketaan, kunnes generoidaan pääteimerkki tai saavutetaan haluttu tekstin pituus.

Vaikka lähestymistapa on käsitteellisesti yksinkertainen ja tulkittava, siihen liittyy merkittäviä laskennallisia haasteita erityisesti kontekstikoon n kasvaessa. Tämän seurauksena lukumatriisista $C \in \mathbb{R}^{|V|^{n-1} \times |V|}$ tulee nopeasti erittäin suuri ja käytännöllisissä sovelluksissa hyvin harva, koska vain pieni osa kaikista mahdollisista konteksteista esiintyy tyypillisessä havaintoaineistossa. Tämä lisää sekä muistin- että laskentatehon tarvetta [15].

Diskreetit esitykset ja frekvenssipohjaiset n -gram-mallit tarjoavat käsitteellisesti selkeän lähtökohdan kielimallinnukselle. Niiden keskeinen rajoite on kuitenkin heikko yleistyskyky harvinaisiin ja havaitsemattomiin konteksteihin astuen esiin niin sanottuna "ulottuvuuksien kirouksena" (engl. *curse of dimensionality*) [2]. Tämän vuoksi on hyödyllistä tarkastella samaa ongelmaa optimointitehtävänä, mikä muodostaa luontevan siirtymän kohti jatkuvia vektoriesityksiä ja parametrisoituja neuroverkko-pohjaisia malleja.

3 Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus

3.1 Upotusvektorit

One-hot-esitys tarjoaa yksikäsitteisen tavan kuvata tokeneita, mutta sen heikkoutena on, ettei se sisällä eksplisiittistä tietoa tokenien välisistä suhteista. Jokaisen tokenin vektorikuvaus on ortogonaalinen kaikkiin muihin nähden, jolloin esimerkiksi sanan *kissa* suhde sanaan *koira* on rakenteellisesti sama kuin sen suhde sanaan *tietokone*. Tämän rajoitteen ylittämiseksi käytetään upotusvektoreita (engl. *embedding vectors*), joissa kukin token kuvataan tiheällä reaaliarvoisella vektorilla.

Olkoon edelleen $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto. Tällöin tokenille v_j määritellään upotus

$$e(v_j) \in \mathbb{R}^d,$$

missä $d \ll |V|$ on upotustilan dimensio.

Upotusvektorit voidaan esittää myös matriisimuodossa. Olkoon

$$E \in \mathbb{R}^{d \times |V|}$$

upotusmatriisi, jonka sarakkeet vastaavat sanaston tokeneita. Tällöin tokenin v_j upotus saadaan one-hot-vektorin $x(v_j)$ avulla muodossa

$$e(v_j) = Ex(v_j).$$

Koska one-hot-vektori sisältää täsmälleen yhden epä-nollan komponentin, tämä kertolasku palauttaa matriisista E juuri tokenia v_j vastaavan sarakkeen.

Upotusvektorien keskeinen etu liittyy siihen, että samankaltaisten tokenien upotukset voivat sijaita lähellä toisiaan upotustilassa. Tällöin tokenien geometrinen läheisyys voi heijastaa niiden semanttista tai syntaktista samankaltaisuutta. Esimerkiksi sanat “kuningas” ja “kuningatar” voivat saada upotukset, jotka ovat lähellä toisiaan, koska ne esiintyvät samankaltaisissa konteksteissa ja jakavat merkityksellisiä piirteitä. Upotusvektorit tarjoavat siten rikkaamman ja joustavamman esityksen tokenien välisistä suhteista kuin one-hot-esitys, jossa kaikki tokenit ovat yhtä kaukana toisistaan.

Käytännön kielimallinnuksessa upotukset opitaan tavallisesti tehtäväkohtaisesti osana laajempaa optimointia. Jos korpus on $W = (w_1, w_2, \dots, w_{|W|})$, voidaan jokainen tokeni $w_t \in V$ korvata sitä vastaavalla upotusvektorilla $e(w_t)$. Tällöin koko korpusta voidaan tarkastella tiheiden vektorien jonona

$$E_W = (e(w_1), e(w_2), \dots, e(w_{|W|})).$$

Tämä esitystapa on erityisen hyödyllinen neuroverkoissa, joissa upotukset muodostavat syötteen myöhemmille kerroksille ja joissa opittavat parametrit voivat hyödyntää tokenien välisiä yhteyksiä yhteisen esitystilan kautta.

Upotusvektorit ovat myös yhteensopivia erikoistokenien kanssa. Esimerkiksi aloitusmerkki $\langle s \rangle$, lopetusmerkki $\langle /s \rangle$ ja tuntematon token $\langle unk \rangle$ voidaan sisällyttää laajennettuun sanastoon jolloin niille voidaan oppia omat upotuksensa samalla tavalla

kuin muillekin tokeneille. Näin myös sekvenssien rakenteelliset elementit voidaan käsitellä yhtenäisesti muun sanaston kanssa.

Upotusvektorit muodostavat siten luonnollisen jatkeen one-hot-esitykselle. Siinä missä one-hot kuvaa tokenin pelkkänä identiteettinä, upotusvektori tarjoaa tiiviin ja optimoitavan esityksen, joka soveltuu tehokkaasti neuroverkkopohjaisiin kielimalleihin.

3.2 Upotusvektorien oppiminen Word2Vec-menetelmällä

Upotusvektorit eivät tavallisesti ole ennalta määrättyjä, vaan ne estimoidaan aineistosta optimointimenetelmän avulla. Tässä suhteessa yksi keskeisimmistä ja vaikutusvaltaisimmista menetelmistä on Word2Vec, jonka Mikolov ym. esittelevät työssään *Efficient Estimation of Word Representations in Vector Space* [21]. Menetelmän keskeinen tieteellinen kontribuutio ei rajoitu siihen, että sanoille opitaan tiheitä vektoriesityksiä, sillä vastaavia ajatuksia oli esitetty neuroverkkopohjaisessa kielimallinnuksessa jo aiemmin. Word2Vecin varsinainen merkitys liittyy ennen kaikkea siihen, että se osoitti tällaisten representaatioiden oppimisen olevan mahdollista laskennallisesti erittäin tehokkaalla tavalla myös hyvin suurissa korpuksissa. Lisäksi se toi esiin, että kontekstuaaliseen ennustamiseen perustuva oppiminen voi johtaa vektoriavaruuksiin, joissa ilmenee kielellisesti mielekkäitä semanttisia ja syntaktisia säännönmukaisuuksia.

Word2Vec perustuu distributionaaliseen hypoteesiin, jonka mukaan sanan merkitys voidaan päätellä sen käyttöympäristöjen perusteella. Toisin sanoen sanat, jotka esiintyvät samankaltaisissa konteksteissa, saavat oppimisprosessissa samankaltaiset vektoriesitykset. Tämä periaate on laajasti koko modernin kieliteknologian taustalla, mutta Word2Vec teki siitä käytännöllisen ja skaalautuvan laskennallisen menetelmän. Menetelmä osoitti, että diskreettien symbolien semanttista rakennetta voidaan mallintaa jatkuvassa vektoriavaruudessa pelkästään paikallisiin konteksteihin perustuvan ennustetehtävän avulla ilman käsin suunniteltuja kielellisiä piirteitä.

Olkon korpus $W = (w_1, w_2, \dots, w_{|W|})$, missä $w_t \in V$ on sanaston V sana ajanhetkellä t . Word2Vec-malleissa tarkastellaan tyypillisesti kunkin sanan ympärille määritettyä paikallista kontekstia, jonka laajuus määräytyy ikkunaparametrilla $c \in \mathbb{N}$. Tällöin ajanhetken t kontekstiksi voidaan määritellä

$$C_t = \{w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}\},$$

olettaen, että indeksit pysyvät korpuksen rajojen sisällä. Mikolov ym. [21] tarkastelevat kahta vaihtoehtoista perusarkkitehtuuria: jatkuvien sanojen pussimallia (*continuous bag-of-words*, CBOW) ja Skip-gram-mallia.

CBOW-mallissa pyritään ennustamaan tarkasteltava keskussana sen kontekstisanojen perusteella. Tavoitteena on siten approksimoida todennäköisyyttä

$$p(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}).$$

Kontekstin sanat projisoidaan ensin vektoriesityksiksi, minkä jälkeen nämä esitykset yhdistetään tavallisesti summaamalla tai keskiarvoistamalla. Muodostettua kontekstivektoria käytetään tämän jälkeen kohdesanan ennustamiseen koko sanaston yli.

CBOW-mallin etuna on sen laskennallinen tehokkuus erityisesti suurilla aineistoilla, koska useiden kontekstisanojen informaatio voidaan tiivistää yhdeksi esitykseksi ennen ennustevaihetta [21].

Skip-gram-mallissa ennustesuunta on päinvastainen. Siinä tavoitteena on ennustaa keskussanan avulla sitä ympäröivät kontekstisanat. Tällöin maksimoidaan todennäköisyydet

$$p(w_{t+j} \mid w_t), \quad -c \leq j \leq c, j \neq 0.$$

Koko korpuksen tasolla vastaava tavoitefunktio voidaan kirjoittaa muodossa

$$\sum_{t=1}^{|W|} \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

Skip-gram-malli osoittautuu erityisen käyttökelpoiseksi silloin, kun tavoitteena on oppia informatiivisia esityksiä myös harvinaisemmille sanoille, sillä kukin havaittu sana toimii useiden kontekstisanojen ennustamisen lähtökohtana [21].

Olkoon w_I syötesana ja w_O kohdesana. Jos syötesanaa vastaava vektoriesitys on v_{w_I} ja kohdesanaa vastaava ulostulovektori on u_{w_O} , voidaan todennäköisyys $p(w_O \mid w_I)$ esittää softmax-funktion avulla muodossa

$$p(w_O \mid w_I) = \frac{\exp(u_{w_O}^\top v_{w_I})}{\sum_{w \in V} \exp(u_w^\top v_{w_I})}.$$

Tässä nimittäjän summa ulottuu koko sanastoon, mikä tekee täsmällisestä optimoinnista laskennallisesti raskasta suurilla sanastoilla. Juuri tämän ongelman ratkaiseminen on yksi Word2Vecin keskeisistä ansioista. Menetelmä teki näkyväksi sen, että suuren sanaston ulostuloprojektio muodostaa vektoriesitysten oppimisessa keskeisen laskennallisen pullonkaulan, ja esitti tähän käytännöllisiä approksimaatioita. Näistä erityisen merkittävä on negatiivinen otanta (*negative sampling*), jossa koko sanaston yli määritellyn softmax-todennäköisyyden optimoinnin sijasta ratkaistaan joukko binäärisiä luokittelutehtäviä.

Olkoon (w_I, w_O) havaittu syöte–kohdesana-pari, jossa w_O esiintyy sanan w_I kontekstissa. Negatiivisessa otannassa valitaan lisäksi K negatiivista sanaa $w_1^-, \dots, w_K^-$ jakaumasta $P_n(w)$, joka on tyypillisesti jokin sanaston sanoille määritelty näytteenottojakauma. Tällöin mallin tavoitteena ei ole enää optimoida todennäköisyyttä $p(w_O \mid w_I)$ koko sanaston yli, vaan maksimoida havaittua paria vastaava binäärinen logistinen tavoite

$$\log \sigma(u_{w_O}^\top v_{w_I}) + \sum_{k=1}^K \log \sigma(-u_{w_k^-}^\top v_{w_I}),$$

missä $\sigma(x) = \frac{1}{1+e^{-x}}$ on logistinen sigmoidifunktio. Ensimmäinen termi kasvattaa havaittuun positiiviseen sana–konteksti-pariin liittyvän pistetulon $u_{w_O}^\top v_{w_I}$ arvoa, kun taas jälkimmäinen termi pienentää satunnaisesti valittujen negatiivisten sanojen $u_{w_k^-}^\top v_{w_I}$ arvoja. Näin malli oppii erottamaan aidot kontekstisanat satunnaisista sanoista ilman, että jokaisella oppimisaskeleella tarvitsisi normalisoida jakaumaa koko sanaston yli.

Koko aineiston yli tavoite voidaan esittää odotusarvomuodossa

$$\sum_{(w_I, w_O) \in D} \left[\log \sigma(u_{w_O}^\top v_{w_I}) + \sum_{k=1}^K \mathbb{E}_{w_k^- \sim P_n} \log \sigma(-u_{w_k^-}^\top v_{w_I}) \right],$$

missä D on havaittujen syöte–konteksti-parien joukko ja K on negatiivisten otantojen määrä. Tällä tavoin alkuperäinen moniluokkainen softmax-optimointi approksimoidaan joukolla logistisia regressiotehtäviä, joissa erotellaan positiiviset havaintoparit negatiivisesti näytteistetyistä pareista. Laskennallinen etu syntyy siitä, että kunkin havaintoparin kohdalla päivitetään vain positiivista kohdesanaa ja K :ta negatiivista sanaa vastaavat parametrit koko sanaston kattavan päivityksen sijasta.

Word2Vec-mallissa opitaan kaksi parametrimatriisia: syötevektorit ja ulostulovektorit. Jokaiselle sanaston sanalle $v_j \in V$ assosioituu siten ainakin yksi tiheä vektoriesitys. Oppimisen jälkeen sanan lopullisena esityksenä käytetään tavallisesti syötevektoria tai vaihtoehtoisesti syöte- ja ulostulovektoreiden yhdistelmää. Näin saadut upotukset heijastavat korpuksessa havaittuja yhteisesiintymisrakenteita siten, että samankaltaisissa käyttöympäristöissä esiintyvät sanat sijoittuvat tyypillisesti lähelle toisiaan upotustilassa.

Mikolov ym. [21] osoittavat lisäksi, että Word2Vecin oppimat vektoriesitykset eivät ainoastaan ryhmittele semanttisesti läheisiä sanoja, vaan niihin syntyy myös lineaarisia relaatiota vastaavia säännönmukaisuuksia. Sanavektorien erotukset voivat vastata esimerkiksi sukupuoleen, lukuun tai taivutusmuotoon liittyviä semanttisia ja syntaktisia eroja, mikä ilmenee analogiatehtävissä. Tämä havainto on menetelmän kannalta erityisen merkittävä, sillä se viittaa siihen, että kontekstuaaliseen ennustamiseen perustuva optimointi tuottaa emergentisti sellaisia vektoriavaruuden rakenteita, jotka eivät ole eksplisiittisesti koodattuja malliin, mutta jotka vastaavat kielellisesti tulkittavia suhteita. Näin Word2Vec osoitti, että jakaumallinen oppiminen ei tuota ainoastaan samankaltaisuusmittaa, vaan voi synnyttää myös abstraktimpia relaatioita mallintavia geometrisia rakenteita.

Vaikka Word2Veciä ei nykyisissä suurissa kielimalleissa tyypillisesti käytetä sellaisenaan erillisenä menetelmänä, sen vaikutus näkyy edelleen usealla tasolla. Ensinnäkin se vakiinnutti distributionaalisen hypoteesin neuroverkkopohjaisen kielimallinuksen keskeiseksi oppimisperiaatteeksi. Toiseksi se toi esiin suuren sanaston ulostuloprojektion laskennallisen haasteen sekä popularisoi tehokkaita approksimointimenetelmiä, kuten negatiivisen otannan. Kolmanneksi se osoitti vakuuttavasti, että yksinkertainenkin kontekstuaalinen ennustetehtävä voi johtaa vektoriavaruuksiin, joissa semanttiset ja syntaktiset suhteet ilmenevät lineaarisina rakenteina. Näistä syistä Word2Vecia voidaan pitää tärkeänä siirtymävaiheena klassisten jakaumallisten sanamallien ja myöhempien syvien neuroverkkojen, mukaan lukien nykyaikaiset suuret kielimallit, välillä.

3.3 N-gram-malli optimointiongelmana

Luvussa 2.2 n -gram-malli esitettiin lukumääriin ja niiden normalisointiin perustuva frekvenssimallina. Sama malli voidaan kuitenkin tulkita myös suurimman uskottavuuden estimointina, mikä tekee yhteyden myöhempiin parametrisiin kielimalleihin näkyväksi.

Olkoon mallin parametreina jokaiselle kontekstille $c = (w_{t-n+1}, \dots, w_{t-1})$ määritelty todennäköisyysjakauma

$$\theta_c = (\theta_{c,1}, \theta_{c,2}, \dots, \theta_{c,|V|}), \quad \sum_{j=1}^{|V|} \theta_{c,j} = 1, \quad \theta_{c,j} \geq 0,$$

missä $\theta_{c,j} = P_\theta(w_t = v_j \mid c)$. Markov-oletuksen nojalla havaintoaineiston $W = (w_1, \dots, w_{|W|})$ uskottavuus voidaan kirjoittaa muodossa

$$P(W; \theta) \approx \prod_{t=1}^{|W|} P_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Kun aineisto ryhmitellään kontekstin c ja sitä seuraavan tokenin v_j mukaan, saadaan

$$P(W; \theta) = \prod_c \prod_{j=1}^{|V|} \theta_{c,j}^{C_{c,j}},$$

missä $C_{c,j}$ on niiden havaintojen lukumäärä, joissa kontekstia c seuraa tokeni v_j .

Koneoppimisessa optimointi esitetään tavallisesti minimointitehtävänä, joten määritellään häviöksi negatiivinen log-uskottavuus

$$J(\theta; W) = -\log P(W; \theta) = -\sum_c \sum_{j=1}^{|V|} C_{c,j} \log \theta_{c,j}.$$

Koska parametrit θ_c esiintyvät vain omaa kontekstiaan vastaavissa termeissä, optimointiongelma hajoaa erillisiksi kontekstikohtaisiksi osatehtäviksi:

$$\min_{\theta_c} -\sum_{j=1}^{|V|} C_{c,j} \log \theta_{c,j} \quad \text{s.e.} \quad \sum_{j=1}^{|V|} \theta_{c,j} = 1.$$

Tämä voidaan ratkaista Lagrangen menetelmällä. Määritellään

$$\mathcal{L}_{\text{Lag}}(\theta_c, \lambda) = \sum_{j=1}^{|V|} C_{c,j} \log \theta_{c,j} + \lambda \left(1 - \sum_{j=1}^{|V|} \theta_{c,j} \right).$$

Derivoimalla $\theta_{c,j}$:n suhteen saadaan

$$\frac{\partial \mathcal{L}_{\text{Lag}}}{\partial \theta_{c,j}} = \frac{C_{c,j}}{\theta_{c,j}} - \lambda = 0,$$

mistä seuraa

$$\theta_{c,j} = \frac{C_{c,j}}{\lambda}.$$

Kun tämä sijoitetaan normalisaatioehtoon

$$\sum_{j=1}^{|V|} \theta_{c,j} = 1,$$

saadaan

$$\lambda = \sum_{k=1}^{|V|} C_{c,k},$$

ja siten

$$\theta_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|V|} C_{c,k}}.$$

Tulos osoittaa, että suurimman uskottavuuden estimaatti on täsmälleen sama kuin luvussa 2.2 määritelty rivikohtaisesti normalisoitu frekvenssimatriisi F . Toisin sanoen klassinen frekvenssipohjainen n -gram-malli voidaan tulkita parametrisen mallin maksimiuskottavuusratkaisuna.

Tämä näkökulma tekee samalla näkyväksi mallin keskeisen rajoitteen: jokaiselle mahdolliselle kontekstille tarvitaan oma erillinen todennäköisyysjakaumansa. Kuten luvussa 2.2 nähtiin, kontekstin pituuden kasvaessa parametrien määrä kasvaa nopeasti, jolloin malli muuttuu harvaksi ja yleistää heikosti harvinaisiin tai havaitsemattomiin konteksteihin. Tästä syystä on luontevaa siirtyä malleihin, joissa ehdolliset todennäköisyydet opitaan jaetun parametrisaation avulla jatkuvassa avaruudessa sen sijaan, että ne talletettaisiin erillisinä taulukoituina jakaumina.

3.4 Monikerroksinen perseptroni seuraavan tokenin ennustajana

Seuraavan tokenin ehdollista jakaumaa voidaan approksimoida parametrisoidulla neuroverkolla. Yksi yksinkertainen ja historiallisesti keskeinen ratkaisu tähän on monikerroksinen perseptroni (engl. *multilayer perceptron*, MLP).

MLP on eteenpäinsyöttävä keinotekoinen neuroverkko, joka koostuu syötekerroksesta, yhdestä tai useammasta piilokerroksesta sekä ulostulokerroksesta. Sen historialliset juuret ovat Frank Rosenblattin perseptronitutkimuksessa [30][31]. Näissä töissä esitettiin varhaisia perseptronimalleja ja kerroksellisia rakenteita, mutta tehokas yleiskäyttöinen menetelmä kaikkien kerrosten parametrien oppimiseen puuttui vielä.

Neuroverkkojen varhaista kehitystä hidasti merkittävästi Marvin Minskyn ja Seymour Papertin teos, jossa analysoitiin yksikerroksisten perseptronien rajoitteita [22]. Teos osoitti, että yksinkertainen perseptroni ei voi ratkaista tiettyjä lineaarisesti erottamattomia ongelmia, kuten XOR-tehtävää. Vaikka kritiikki kohdistui ensisijaisesti yksikerroksisiin malleihin, sen vaikutus ulottui laajemmin neuroverkko-tutkimukseen ja osaltaan vähensi kiinnostusta monikerroksisiin verkkoihin useiden vuosien ajaksi.

Monikerroksisten verkkojen tehokkaan opettamisen kannalta keskeinen teoreettinen lähtökohta syntyi Seppo Linnainmaan työssä, jossa esitettiin käänteisen suunnan automaattinen derivointi menetelmä [18]. Paul Werbos puolestaan sovelsi tätä ajatusta neuroverkkoihin väitöskirjassaan ja myöhemmin artikkelissaan, jossa termi vastavirta-algoritmi (engl. *backpropagation*) vakiintui [44][43].

Monikerroksisten eteenpäinsyöttävien verkkojen läpimurto tapahtui kuitenkin vasta, kun David Rumelhart, Geoffrey Hinton ja Ronald Williamsin teos popularisoi

vastavirta-algoritmin[32]. Tämä työ teki MLP:stä käytännöllisen gradienttipohjaisesti opetettavan funktion approksimaattorin ja loi perustan myöhemmälle syväoppimiselle.

Matemaattisesti MLP voidaan ymmärtää funktion approksimaattorina, joka muodostaa kuvauksen syöteavaruudesta tulosteavaruuteen. Tämä kuvaus toteutetaan yhdistettynä funktiona, joka koostuu peräkkäisistä affineista muunnoksista ja niitä seuraavista epälineaarisista aktivaatiofunktioista. Tämän ansiosta MLP kykenee mallintamaan sellaisia syöteen ja tulosteen välisiä riippuvuuksia, joita lineaarinen malli ei voi esittää. Mallin toimintaa ohjaavat opittavat parametrit, tyypillisesti painomatriisit ja bias-vektorit, joiden arvot estimoidaan datasta minimoimalla valittua kohdefunktiota numeerisilla optimointimenetelmillä, kuten gradienttimenetelyllä [14].

Olkoon c tarkasteltava konteksti ja $x(c) \in \mathbb{R}^{d_0}$ sitä vastaava syötevektori. Monikerroksinen perceptroni muuntaa tämän syöteen vaiheittain piirre-esitykseksi soveltamalla peräkkäisiä affiinisia muunnoksia ja epälineaarisia aktivaatioita. Jos mallissa on L piilokerrosta, voidaan sen laskenta esittää muodossa

$$\begin{aligned} h^{(0)} &= x(c), \\ a^{(\ell)} &= W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, & \ell = 1, \dots, L, \\ h^{(\ell)} &= \phi^{(\ell)}(a^{(\ell)}), & \ell = 1, \dots, L, \end{aligned}$$

missä $W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ on kerroksen ℓ painomatriisi, $b^{(\ell)} \in \mathbb{R}^{d_\ell}$ sen bias-vektori, $a^{(\ell)} \in \mathbb{R}^{d_\ell}$ affiinisien muunnosten tulos ja $h^{(\ell)} \in \mathbb{R}^{d_\ell}$ kerroksen ℓ piiloesitys. Funktio $\phi^{(\ell)}$ on kerroksen epälineaarinen aktivaatiofunktio.

Viimeisestä piilokerroksesta muodostetaan ulostulon normalisoimattomat pistemäärät eli logitit

$$z = W^{(\text{out})} h^{(L)} + b^{(\text{out})},$$

missä $W^{(\text{out})} \in \mathbb{R}^{|V| \times d_L}$, $b^{(\text{out})} \in \mathbb{R}^{|V|}$ ja $z \in \mathbb{R}^{|V|}$. Vektorin z komponentti z_j vastaa sanaston tokenin v_j normalisoimatonta pistemäärää. Näistä muodostetaan ehdollinen todennäköisyysjakauma softmax-funktiolla:

$$p_\theta(w_t = v_j \mid c) = \frac{\exp(z_j)}{\sum_{k=1}^{|V|} \exp(z_k)}, \quad j = 1, \dots, |V|.$$

MLP muuntaa kontekstin ensin yhden tai usean kerroksen kautta abstraktiksi piirre-esitykseksi, jonka perusteella se laskee jokaiselle sanaston tokenille pistemäärän ja normalisoi nämä pistemäärät todennäköisyysjakaumaksi.

Mallin oppiminen perustuu samaan negatiiviseen log-uskottavuuden minimointiin kuin n -gram-mallissa:

$$J(\theta) = -\frac{1}{|W|} \sum_{t=1}^{|W|} \log p_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Erona perinteiseen frekvenssipohjaiseen n -gram-malliin, joka estimoi jokaiselle mahdolliselle diskreetille kontekstille oman erillisen parametrivektorinsa, MLP approksimoi todennäköisyyksiä jatkuvassa avaruudessa. Koska erilliset tokenit projisoidaan matalaulotteisiksi tiheiksi upotusvektoreiksi $v \in \mathbb{R}^d$, malli oppii esittämään sanojen semanttisia ja syntaktisia samankaltaisuuksia vektoriesitysten avulla. MLP:n

approksimoima funktio on jatkuva ja tyypillisesti sileä siinä mielessä, että pienet muutokset syöteavaruudessa voivat johtaa samankaltaisiin ulostulojakaumiin. Tästä syystä malli kykenee yleistämään myös harvinaisiin tai aiemmin havaitsemattomiin konteksteihin hyödyntämällä sellaisten koulutusdatassa esiintyneiden kontekstien rakennetta, joiden esitykset sijaitsevat lähellä niitä yhteisessä avaruudessa [2]. Tämä lieventää klassisia n -gram-malleja vaivaavaa nollatodennäköisyyksien ongelmaa ilman erillisiä tasoitusmenetelmiä.

Frekvenssipohjaista mallia koskevat tekniset rajoitteet, kuten eksponentiaalinen muistitarpeen kasvu ja lukumatriisin harvuus, voidaan siten osittain ratkaista MLP-mallin avulla. MLP-rakenteessa opittavat painomatriisit ja bias-vektorit jakautuvat kaikkien kontekstien kesken. Siten jokainen harjoitusnäyte päivittää koko neuroverkon parametreja, mikä parantaa myös sellaisten lauseiden ennustamista, joita ei ole esiintynyt opetusdatassa, mutta jotka jakavat samankaltaisia piirteitä opittujen esitysten kautta. Tämä tekee MLP-pohjaisesta n -gram-mallista huomattavasti joustavamman, muistikapasiteetiltaan tehokkaamman ja yleistyskykyisemmän kuin perinteinen frekvenssiperusteinen taulukkolukuesitys [2, 14, 15].

4 Gradienttimenetelmä

Gradienttimenetelmä ja sen stokastinen muunnelmä ovat ensimmäisen kertaluvun optimointimenetelmiä, joissa kohdefunktion gradienttia hyödynnetään parametrivektorin päivittämisessä. Tarkastellaan minimointitehtävää

$$\min_{\theta \in \mathbb{R}^d} J(\theta),$$

missä $J : \mathbb{R}^d \rightarrow \mathbb{R}$ on jatkuvasti derivoituva kohdefunktio ja $\theta \in \mathbb{R}^d$ sen parametrivektori. Gradienttimenetelmän perusajatus perustuu kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Olkoon $\theta_t \in \mathbb{R}^d$ ajanhetken t parametrivektori ja $v \in \mathbb{R}^d$ pieni siirtymävektori. Taylorin lauseen nojalla saadaan

$$J(\theta_t + v) = J(\theta_t) + \langle \nabla J(\theta_t), v \rangle + o(\|v\|_2), \quad \text{kun } \|v\|_2 \rightarrow 0.$$

Tästä seuraa, että funktion arvon paikallista muutosta approksimoi ensisijaisesti lineaarinen termi $\langle \nabla J(\theta_t), v \rangle$. Rajoittamalla siirtymän pituutta ehdolla $\|v\|_2 \leq \varepsilon$ saadaan minimointitehtävä

$$\min_{\|v\|_2 \leq \varepsilon} \langle \nabla J(\theta_t), v \rangle.$$

Olettamalla, että $\nabla J(\theta_t) \neq 0$, Cauchy–Schwarzin epäyhtälöstä seuraa arvio

$$\langle \nabla J(\theta_t), v \rangle \geq -\|\nabla J(\theta_t)\|_2 \|v\|_2 \geq -\varepsilon \|\nabla J(\theta_t)\|_2.$$

Yhtäsuuruus toteutuu, kun vektori v on vastakkaissuuntainen gradienttiin nähden. Näin ollen lineaarisen approksimaation kannalta optimaalinen siirtymä on

$$v^* = -\frac{\varepsilon}{\|\nabla J(\theta_t)\|_2} \nabla J(\theta_t).$$

Optimaalinen siirtymä v^* osoittaa, että funktion arvo pienenee paikallisesti nopeimmin negatiivisen gradientin suuntaan. Uusi parametriarvo saadaan asettamalla $\theta_{t+1} = \theta_t + v^*$. Käytännössä normalisointitermi $\varepsilon/\|\nabla J(\theta_t)\|_2$ korvataan usein suoraan oppimisnopeudella $\eta > 0$ (engl. *learning rate*), jolloin askelpituus skaalautuu luonnostaan gradientin suuruuden mukaan. Tämä johtaa gradienttimenetelmän iteraatiiviseen päivityssääntöön

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

Koneoppimisessa kohdefunktio muodostetaan tyypillisesti empiirisenä riskinä:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; z_i),$$

missä $z_1, \dots, z_N$ ovat opetusdatan näytteitä ja $\ell(\theta; z_i)$ on havaintoon z_i liittyvä häviöfunktio. Tällöin täysi gradientti on

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(\theta; z_i).$$

Kielimallien opetuksessa käytettävät aineistot ovat usein erittäin suuria, minkä vuoksi täyden gradientin laskeminen jokaisella iteraatiolla on laskennallisesti kallista. Tämän vuoksi käytetään stokastista gradienttimenetelmää (engl. *stochastic gradient descent*, SGD), jossa täysi gradientti korvataan satunnaisesti poimitun mini-erän perusteella muodostetulla gradienttiestimaatilla [29, 4, 5].

Olkoon $B_t \subset \{1, \dots, N\}$ hetkellä t muodostettu satunnainen mini-erä, jonka koko on $|B_t| = b$. Tällöin gradienttiestimaatti g_t määritellään muodossa

$$g_t = \frac{1}{b} \sum_{i \in B_t} \nabla \ell(\theta_t; z_i),$$

ja SGD:n päivityssääntö on

$$\theta_{t+1} = \theta_t - \eta g_t.$$

Jos mini-erä valitaan hetkellä t tasaisesti kaikkien b -alkioisten indeksijoukkojen joukosta, gradienttiestimaatti on ehdollisen odotusarvon mielessä harhaton [5]

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t).$$

Stokastinen gradientti vastaa siten keskimäärin täyttä gradienttia, vaikka yksittäisen päivitys sisältää satunnaisuudesta johtuvaa kohinaa. Tämä kohina estää yleensä kohdefunktion arvon monotonisen pienenemisen, mutta mini-erien käyttö lieventää laskennallisia haasteita [4, 14].

Sekä gradienttimenetelmän että SGD:n päivitykset perustuvat kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Tämä approksimaatio ei kuitenkaan sellaisenaan takaa kohdefunktion arvon pienenemistä. Tällaisen takuun johtamiseksi tarvitaan gradientin säännöllisyyttä koskevia lisäoletuksia.

Määritelmä 1. Jatkuvasti derivoituva funktio $J : \mathbb{R}^d \rightarrow \mathbb{R}$ on L -sileä, jos sen gradientti ∇J on L -Lipschitz, eli kaikilla $\theta, \theta' \in \mathbb{R}^d$ pätee

$$\|\nabla J(\theta) - \nabla J(\theta')\|_2 \leq L\|\theta - \theta'\|_2.$$

Olettaen, että kohdefunktio on L -sileä, sen muutokselle voidaan johtaa neliöllinen yläraja.

Lemma 1 (Neliöllinen yläraja). *Olkoon $J : \mathbb{R}^d \rightarrow \mathbb{R}$ L -sileä funktio. Tällöin kaikilla $\theta, h \in \mathbb{R}^d$ pätee*

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2} \|h\|_2^2.$$

Todistus. Analyysin peruslauseesta seuraa, että

$$J(\theta + h) - J(\theta) = \int_0^1 \langle \nabla J(\theta + th), h \rangle dt.$$

Lisäämällä ja vähentämällä termi $\langle \nabla J(\theta), h \rangle$ integraalin sisällä saadaan

$$\begin{aligned} J(\theta + h) - J(\theta) &= \int_0^1 \langle \nabla J(\theta), h \rangle dt \\ &\quad + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt \\ &= \langle \nabla J(\theta), h \rangle + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt. \end{aligned}$$

Cauchy–Schwarzin epäyhtälön ja L -Lipschitz-oletuksen perusteella integraalissa oleva termi voidaan arvioida ylhäältä:

$$\begin{aligned}\langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle &\leq \|\nabla J(\theta + th) - \nabla J(\theta)\|_2 \|h\|_2 \\ &\leq Lt \|h\|_2^2.\end{aligned}$$

Integroimalla saadaan

$$\begin{aligned}\int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt &\leq \int_0^1 Lt \|h\|_2^2 dt \\ &= \frac{L}{2} \|h\|_2^2.\end{aligned}$$

Näin ollen

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2} \|h\|_2^2.$$

□

Neliöllinen yläraja tarjoaa keskeisen työkalun optimointimenetelmien analysointiin. Soveltamalla sitä gradienttimenetelmän päivitysaskeleeseen, jossa parametreja siirretään negatiivisen gradientin suuntaan askelpituudella $\eta > 0$, saadaan taattua kohdefunktion arvon pieneneminen kullakin iteraatiolla. Tämä tulos tunnetaan laskeutumislemmanna.

Lemma 2 (Laskeutuslemma). *Olko $J : \mathbb{R}^d \rightarrow \mathbb{R}$ L -sileä funktio ja $0 < \eta \leq 1/L$. Tällöin gradienttimenetelmän päivityksellä*

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

pätee

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2} \|\nabla J(\theta_t)\|_2^2.$$

Todistus. Sovelletaan neliöllistä ylärajaa (lemma 1) sijoittamalla $\theta = \theta_t$ ja $h = -\eta \nabla J(\theta_t)$, jolloin $\theta + h = \theta_{t+1}$. Tällöin saadaan

$$\begin{aligned}J(\theta_{t+1}) &\leq J(\theta_t) + \langle \nabla J(\theta_t), -\eta \nabla J(\theta_t) \rangle + \frac{L}{2} \|\eta \nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \|\nabla J(\theta_t)\|_2^2 + \frac{L\eta^2}{2} \|\nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|_2^2.\end{aligned}$$

Koska $0 < \eta \leq 1/L$, pätee $1 - \frac{L\eta}{2} \geq \frac{1}{2}$, mistä seuraa

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2} \|\nabla J(\theta_t)\|_2^2.$$

□

Lemma osoittaa, että kohdefunktion arvo pienenee jokaisessa ei-stationaarisessa pisteessä, kun oppimismen nopeus valitaan ehdon $0 < \eta \leq 1/L$ mukaisesti. Käytännössä oppimismen nopeuden valinta ei kuitenkaan ole suoraviivaista, sillä kohdefunktion kaa-revuus voi vaihdella merkittävästi eri parametrisuunnissa. Seuraavassa alaluvussa tarkastellaan tämän vaihtelun aiheuttamia haasteita optimoinnille.

4.1 Häiriöalttius ja optimoinnin haasteet

Stokastisen gradienttimenetelmän keskeinen haaste liittyy siihen, että kohdefunktion lokaali kaarevuus voi vaihdella huomattavasti eri parametrisuunnissa. Tätä ilmiötä kutsutaan numeeriseksi häiriöalttiudeksi (engl. *ill-conditioning*). Olkoon kohdefunktio J L -sileä ja kahdesti jatkuvasti derivoituva. Tällöin pisteen θ_t ympäristössä pätee toisen kertaluvun Taylorin approksimaatio

$$J(\theta_t + \Delta\theta) \approx J(\theta_t) + \langle \nabla J(\theta_t), \Delta\theta \rangle + \frac{1}{2} \langle \Delta\theta, \nabla^2 J(\theta_t) \Delta\theta \rangle.$$

Merkitään Hessin matriisia pisteessä θ_t symbolilla

$$H_t = \nabla^2 J(\theta_t).$$

Koska H_t on symmetrinen, sen ominaisarvot ovat reaalisia. Oletetaan tässä, että lokaali kvadraattinen malli on konvekksi, eli $H_t \succeq 0$. Tällöin ominaisarvot voidaan järjestää muodossa

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0,$$

missä λ_1 vastaa suurinta ja λ_d pienintä paikallista kaarevuutta.

Koska J on L -sileä ja kahdesti jatkuvasti derivoituva, Hessin operaattorinormille pätee

$$\|H_t\|_2 \leq L.$$

Koska H_t on symmetrinen, tästä seuraa matriisiepäyhtälö

$$-LI \preceq H_t \preceq LI,$$

missä $\preceq$ on Löwner-järjestys. Erityisesti lokaalisti konveksisessa tapauksessa $H_t \succeq 0$ saadaan

$$0 \preceq H_t \preceq LI,$$

mistä seuraa

$$\lambda_1 \leq L.$$

Tämä rajoittaa oppimisnopeuden kokoa, sillä liian suuri askel voi johtaa epävakaiseen käyttäytymiseen jyrkimmissä suunnissa. Keskeisempi ongelma on kuitenkin se, että gradienttimenetelmä käyttää samaa oppimisnopeutta kaikissa parametrisuunnissa. Jos kohdefunktion lokaali kaarevuus vaihtelee suunnasta toiseen voimakkaasti, oppimisnopeus on valittava suurimman kaarevuuden ehdoilla. Tällöin menetelmä pysyy vakaana jyrkissä suunnissa, mutta etenee samalla hitaasti loivissa suunnissa [3, 23].

Tätä voidaan havainnollistaa tarkastelemalla funktion käyttäytymistä stationaarisen pisteen θ^* ympärillä, missä $\nabla J(\theta^*) = 0$. Jos J on kahdesti jatkuvasti derivoituva, gradienttia voidaan pisteen θ^* läheisyydessä approksimoida lineaarisesti:

$$\nabla J(\theta_t) \approx \nabla^2 J(\theta^*)(\theta_t - \theta^*).$$

Merkitään $H^* = \nabla^2 J(\theta^*)$ ja $e_t = \theta_t - \theta^*$. Tällöin gradienttimenetelmän päivitys saa lokaalisti approksimaation

$$e_{t+1} \approx (I - \eta H^*)e_t.$$

Koska H^* on symmetrinen, se voidaan diagonalisoida ortonormaalissa ominaisvektorikannassa. Oletetaan lisäksi, että θ^* on lokaali minimi, jolloin $H^* \succeq 0$. Jos ominaisarvot järjestetään muotoon

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0,$$

niin virheen komponentit toteuttavat approksimatiivisesti relaation

$$e_{t+1}^{(k)} \approx (1 - \eta \lambda_k) e_t^{(k)}, \quad k = 1, \dots, d.$$

Sama oppimisnopeus η vaikuttaa siis eri suuntiin eri tavoin. Suurta ominaisarvoa vastaavissa suunnissa tekijä $1 - \eta \lambda_k$ muuttuu nopeasti, joten vakaus edellyttää pientä oppimisnopeutta. Jos η valitaan lähelle arvoa $1/\lambda_1$, jyrkimmässä suunnassa pätee $1 - \eta \lambda_1 \approx 0$, jolloin eteneminen on nopeaa. Loivimmassa suunnassa pätee tällöin puolestaan $1 - \eta \lambda_d \approx 1$, mikä merkitsee hidasta supistumista. Menetelmä voi siten olla jyrkissä suunnissa lähes optimaalinen mutta loivissa suunnissa samalla hyvin hidas.

Näin gradienttimenetelmä voi joutua tilanteeseen, jossa oppimisnopeus on jyrkien suuntien kannalta lähes suurin sallittu, mutta loivissa suunnissa eteneminen on silti hidasta. Juuri tämä kaarevuuden epätasaisuus tekee menetelmästä tehottoman huonosti ehdollistuneissa ongelmissa. Ilmiötä voidaan kuvata Hessin matriisin ehtoluvulla

$$\kappa = \frac{\lambda_1}{\lambda_d},$$

kun $\lambda_d > 0$. Positiivisesti definiitillä kvadraattisella kohdefunktiolla gradienttimenetelmän suppenemisnopeus riippuu oleellisesti ehtoluvusta. Suuri ehtoluku tarkoittaa, että kaarevuus vaihtelee huomattavasti eri suunnissa, jolloin yksi globaali oppimisnopeus soveltuu väistämättä huonosti ainakin osaan suunnista [24, 33].

Vaikka analyysi perustuu lokaaliin toisen kertaluvun approksimaatioon, sen johtopäätös on yleisempi. Jos kohdefunktion paikallinen kaarevuus vaihtelee voimakkaasti parametriavaruuden eri suunnissa, yksi kaikissa suunnissa käytettävä oppimisnopeus ei pysty mukautumaan tehokkaasti näihin eroihin. Tällöin oppimisnopeus on valittava jyrkimpien suuntien ehdoilla, mikä hidastaa etenemistä loivemmissa suunnissa [24, 3]. Tämä motivoi menetelmiä, joissa gradienttipäivityksiä skaalataan tai mukautetaan parametrikohteisesti esimerkiksi gradienttihistorian perusteella.

4.2 Adaptiiviset optimointimenetelmät

Edellisessä alaluvussa todettiin, että stokastinen gradienttimenetelmä käyttää samaa globaalia oppimisnopeutta kaikissa parametrisuunnissa. Tämä voi olla tehottomasta erityisesti silloin, kun kohdefunktion lokaali kaarevuus tai gradienttien mitta-kaava vaihtelee voimakkaasti parametrien välillä. Tällöin vakaa oppimisnopeus on

valittava jyrkkien suuntien ehdoilla, mikä hidastaa etenemistä loivemmissa suunnissa. Adaptiivisten optimointimenetelmien keskeinen idea on lieventää tätä ongelmaa skaalaamalla päivitystä parametrikohteisesti gradienttihistorian perusteella.

Yleinen adaptiivinen menetelmä voidaan kirjoittaa muodossa

$$\theta_{t+1} = \theta_t - \eta_t D_t g_t,$$

missä g_t on hetkellä t laskettu stokastinen gradientti, $\eta_t > 0$ on globaali oppimisnopeus ja $D_t \in \mathbb{R}^{d \times d}$ on tavallisesti diagonaalinen, positiivinen skaalausmatriisi, joka riippuu aiemmista gradienteista. Tällöin kunkin parametrin efektiivinen askelpituus määräytyy sekä globaalin oppimisnopeuden että kyseisen parametrin gradienttihistoriaan perustuvan skaalauksen perusteella. Intuitiivisesti komponentit, joiden gradientit ovat usein suuria, voidaan päivittää varovaisemmin, kun taas pienempiä tai harvinaisempia gradientteja saaville komponenteille voidaan sallia suhteellisesti suurempia askelia.

Adaptiivisten menetelmien kehitys syväoppimisessa on rakentunut erityisesti AdaGradin, RMSPropin ja Adamin ympärille. AdaGrad tuo esiin parametrikohteisen oppimisnopeuden perusidean, RMSProp korjaa AdaGradin käytännöllistä heikkoutta ja Adam yhdistää adaptiivisen skaalaamisen sekä momentumin. Modernissa neuroverkkojen ja erityisesti transformer-arkkitehtuurien opetuksessa myös AdamW on saavuttanut vakiintuneen aseman [12, 41, 16, 20, 42].

Momentum. Ennen puhtaasti adaptiivisia menetelmiä stokastisen gradienttimeetelmän keskeisimmäksi laajennukseksi vakiintui momentum- eli liikemäärämenetelmä [25, 40]. Menetelmän tarkoituksena on nopeuttaa optimointia loivilla alueilla sekä vaimentaa oskillaatiota suuntiin, joissa kohdefunktion kaarevuus on suurta. Tämä saavutetaan lisäämällä päivitykseen liukuma aiemmista siirtymistä:

$$v_t = \gamma v_{t-1} + \eta g_t,$$

missä $\gamma \in [0, 1)$ on momentum-kerroin. Parametripäivitys tehdään kertyneen nopeusvektorin v_t suunnassa:

$$\theta_{t+1} = \theta_t - v_t.$$

Vaihtoehtoisesti momentum voidaan formuloida eksponentiaalisesti painotettuna gradienttien liukuvana keskiarvona $m_t = \beta m_{t-1} + (1 - \beta)g_t$, jolloin päivitys on muotoa $\theta_{t+1} = \theta_t - \eta m_t$. Tämä jälkimmäinen esitysmuoto toimii suoraan perustana Adamin ensimmäisen momentin laskennalle.

AdaGrad. AdaGrad (*adaptive gradient*) oli ensimmäisiä laajasti tunnettuja menetelmiä, joissa oppimisnopeus mukautetaan parametrikohteisesti gradienttihistorian perusteella [12]. Menetelmässä ylläpidetään neliöityjen gradienttien kumulatiivista summaa

$$s_t = \sum_{\tau=1}^t g_{\tau}^2,$$

missä potenssi- ja neliöjuurioperaatiot sekä jakolasku tulkitaan komponenttikohtaisesti. Päivitys kirjoitetaan muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon},$$

missä $\varepsilon > 0$ on pieni vakio numeerisen stabiiliuden varmistamiseksi.

AdaGradin keskeinen ominaisuus on, että parametreille, joiden gradientit ovat toistuvasti suuria, kertyy nopeasti suuri nimittäjä, jolloin niiden efektiivinen oppimisnopeus pienenee. Vastaavasti parametreille, joiden gradientit ovat harvinaisia tai pieniä, voivat saada suhteellisesti suurempia päivityksiä. Tämän vuoksi menetelmä soveltuu erityisesti tilanteisiin, joissa gradientit ovat harvoja. AdaGradin keskeinen heikkous on kuitenkin se, että kertymä s_t kasvaa monotonisesti. Tämän seurauksena askelpituudet voivat ajan myötä pienentyä liikaa ja optimointi hidastua olennaisesti.

RMSProp. RMSProp (*root mean square propagation*) kehitettiin käytännölliseksi korjaukseksi AdaGradin liian nopeasti pieneneville askelpituuksille. Menetelmää ei alun perin esitelty vertaisarvioidussa tieteellisessä artikkelissa, vaan se tuotiin julkisuuteen Geoffrey Hintonin pitämällä Coursera-verkkokurssilla [41]. RMSPropin toimintaperiaate on sittemmin dokumentoitu laajasti alan perusteoksissa [14].

Sen sijaan, että neliöityjä gradientteja summattaisiin koko optimoinnin ajalta, RMSProp käyttää eksponentiaalisesti painotettua liukuvaa keskiarvoa

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2,$$

missä $\rho \in (0, 1)$ on muistiparametri. Päivityssäntö on

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

RMSProp säilyttää AdaGradin perusintuition parametrikohtaisesta skaalaamisesta, mutta käyttää pitkän aikavälin kertymän sijasta paikallisempaa estimaattia gradienttien mittakaavasta. Tämä estää oppimisnopeuden systemaattisen pienentymisen ja tekee menetelmästä käytännössä käyttökelpoisemman syvien neuroverkkojen opetuksessa [14].

Adam. Adam (*adaptive moment estimation*) yhdistää kaksi ideaa: gradientin ensimmäisen momentin eli suunnatun liukuvan keskiarvon sekä neliöityjen gradienttien toisen momentin eli liukuvan keskiarvon [16]. Menetelmässä määritellään rekursiot

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \end{aligned}$$

missä $\beta_1, \beta_2 \in (0, 1)$ ovat muistiparametreja. Koska m_t ja v_t alustetaan tavallisesti nolllaksi, ne ovat alkuvaiheessa harhaisia kohti nolllaa. Tämän vuoksi käytetään harhakorjattuja estimaatteja

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}. \end{aligned}$$

Päivitys saa tällöin muodon

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Adamia voidaan tulkita menetelmäksi, jossa m_t vakauttaa päivityssuuntaa momentum-tyyppisesti, kun taas v_t skaalaa askelpituutta parametrikohtaisesti gradienttien havaittuun mittakaavaan nähden. Tämä tekee menetelmästä käytännössä usein tehokkaan ja suhteellisen robustin oppimisnopeuden valinnalle. Näistä syistä Adamista on tullut yksi käytetyimmistä optimointimenetelmistä syväoppimisessa.

Adam ei kuitenkaan ole ongelmaton. On havaittu, että adaptiiviset menetelmät eivät kaikissa tilanteissa tuota yhtä hyvää yleistymistä kuin huolellisesti viritetty SGD momentumilla [45]. Lisäksi Adamin alkuperäinen algoritmi voi tietyillä gradienttisekvensseillä epäonnistua konvergenssissa, minkä vuoksi sen konvergenssianalyysiä on myöhemmin tarkennettu esimerkiksi AMSGrad-menetelmän avulla [28].

Adaptiivisuutta ei siten tule tulkita yleispätevänä ratkaisuna kaikkiin optimoinnin haasteisiin, vaan käytännöllisenä keinona muokata päivityksen geometriaa ja vähentää oppimisnopeuden herkkyyttä.

AdamW. AdamW on Adamin muunnos, jossa painojen kutistaminen (*weight decay*) erotetaan adaptiivisesta gradienttipäivityksestä [20]. Tavallisessa L_2 -reguloinnin ja Adamin yhdistelmässä regularisaatiotermi sekoittuu adaptiiviseen skaalaukseen tavalla, joka ei vastaa puhdasta painojen kutistamista. AdamW:ssä parametrinäpäivitys kirjoitetaan tyypillisesti muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta \lambda \theta_t,$$

missä $\lambda \geq 0$ säätelee painojen kutistamisen voimakkuutta.

Tämä erotus on osoittautunut käytännössä tärkeäksi erityisesti syvien neuroverkkojen opetuksessa [20]. AdamW:tä pidetäänkin usein tarkoituksenmukaisempaan valintaan kuin alkuperäistä Adamia silloin, kun mallin opetuksessa käytetään eksplisiittistä painojen kutistamista. Transformer-arkkitehtuurien opetuksessa Adam-tyyppiset adaptiiviset menetelmät ovat yleisesti käytettyjä [42].

Lopuksi on syytä huomata, että hyvin suurissa kielimalleissa myös optimointimenetelmän muistikustannus muodostuu olennaiseksi. Adam ylläpitää jokaiselle parametrille ensimmäisen ja toisen momentin estimaatit, minkä vuoksi optimointitila voi kuluttaa huomattavan määrän muistia. Tämän vuoksi on kehitetty muistitehokkaampia adaptiivisia menetelmiä, kuten Adafactor, joissa pyritään säilyttämään adaptiivisen skaalaamisen keskeiset edut pienemmällä muistinkulutuksella [37].

Tässä tutkielmassa tarkastelun painopiste rajataan kuitenkin AdaGrad-, RMS-Prop- ja Adam-tyyppisiin menetelmiin, sillä ne muodostavat historiallisesti ja käsitteellisesti keskeisen perustan myöhemmille adaptiivisille optimointialgoritmeille.

5 Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille

Luvuissa 2 ja 3 kielimallinnusta tarkasteltiin seuraavan tokenin ehdollisen todennäköisyyden estimointina. Olkoon V tokenisanasto ja $\Delta(V)$ sanaston yli määriteltyjen todennäköisyysjakaumien joukko. Kielimalli voidaan tällöin kuvata funktiona, joka muuntaa aiemmin havaitun tokenikontekstin seuraavan tokenin todennäköisyysjakaumaksi.

n -gram-mallissa kontekstin pituus on ennalta määrätty. Jos mallin käyttämän kontekstin pituus on m , sitä voidaan kuvata funktiona

$$f_{\text{ngram}}: V^m \rightarrow \Delta(V).$$

Mallin antama jakauma seuraavalle tokenille riippuu siis täsmälleen m :stä edeltävästä tokenista. n -gram-mallin tapauksessa $m = n - 1$, jolloin

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Myös luvussa 3 tarkasteltu MLP-pohjainen kielimalli käyttää kiinteän pituista kontekstia, vaikka sen parametrisoitu esitystapa mahdollistaa frekvenssipohjaista n -gram-mallia paremman yleistämisen.

Nykyaikaiset suuret kielimallit (engl. *large language models*, LLM) perustuvat samaan autoregressiiviseen ennustetehtävään, mutta niiden käsittelemän kontekstin pituus ei ole yksittäisen ennustefunktion kannalta kiinteä. Olkoon L mallin suurin sallittu kontekstinpituus. Dekooderipohjainen Transformer-kielimalli voidaan kuvata funktiona

$$f_{\text{Tr}}: \bigcup_{m=0}^L V^m \rightarrow \Delta(V).$$

Malli voi siten muodostaa seuraavan tokenin jakauman millä tahansa enintään L tokenin pituisella kontekstilla. Käytännössä suurin kontekstinpituus on edelleen arkkitehtuurin ja laskentaresurssien rajoittama, mutta mallin parametrien ei tarvitse olla sidottuja yhteen tiettyyn kontekstin pituuteen.

Tässä luvussa tarkastellaan erityisesti dekodeeripohjaista Transformer-arkkitehtuuria, joka muodostaa useiden generatiivisten kielimallien, kuten GPT-malliperheen, matemaattisen perustan. Transformer esiteltiin alun perin Vaswanin ym. työssä *Attention Is All You Need* vuonna 2017 [42]. Alkuperäinen arkkitehtuuri oli konekäännökseen suunniteltu enkooderi–dekooderi-rakenne. Pian tämän jälkeen esitettiin, että pelkkään Transformer-dekooderiin perustuva autoregressiivinen kielimalli voidaan esikouluttaa suurilla tekstiaineistoilla ja hienosäätää useisiin kielenkäsittelytehtäviin [27].

5.1 Itsehuomio

Transformer-mallin keskeinen mekanismi on itsehuomio (engl. *self-attention*). Sen avulla tokenin esitys voidaan muodostaa koko sitä edeltävän käytettävissä olevan

kontekstin perusteella sen sijaan, että mallissa valittaisiin vain kiinteä määrä viimeisiä tokeneita. Toisin sanoen kontekstin tokenien merkitys ei määräydy pelkästään niiden etäisyyden perusteella, vaan malli voi painottaa ennustettavan tokenin kannalta olennaisia aiempia tokeneita.

Itsehuomiossa jokaisen tokenin esityksestä muodostetaan kysely-, avain- ja arvovektorit. Tarkasteltavan tokenin kyselyvektoria verrataan kontekstin tokenien avainvektoreihin pistetulolla, josta muodostuu huomiopisteet. Huomiopisteet muunnetaan softmax-funktion avulla painoiksi, joilla kontekstin tokenien arvovektorit yhdistetään uuden tokeniesityksen muodostamiseksi.

Kyselyvektoria voidaan tulkita kuvauksena siitä, millaiseen kontekstissa esiintyvään informaatioon tokenin esitys kohdistuu. Avainvektoria voidaan vastaavasti tulkita kuvauksena informaatiosta, jonka perusteella muut tokenit voivat kohdistaa siihen huomiota.

Olkoon tarkasteltavan tokenijonon pituus T , ja olkoon $h_i \in \mathbb{R}^d$ position i tokenia vastaava esitys. Näistä tokeniesityksistä muodostetaan matriisi

$$H = \begin{bmatrix} h_1^\top \\ h_2^\top \\ \vdots \\ h_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

Matriisin H kukin rivi sisältää yhden tokenin d -ulotteisen esityksen.

Itsehuomiomekanismi muodostaa jokaisesta tokeniesityksestä kolme erilaista lineaarista projektiota: kyselyn (engl. *query*), avaimen (engl. *key*) ja arvon (engl. *value*). Olkoot

$$\begin{aligned} W_Q &\in \mathbb{R}^{d \times d_k}, \\ W_K &\in \mathbb{R}^{d \times d_k}, \\ W_V &\in \mathbb{R}^{d \times d_v} \end{aligned}$$

opittavia painomatriiseja. Kysely-, avain- ja arvomatriisit saadaan kertomalla tokeniesitysmatriisi näillä painomatriiseilla:

$$\begin{aligned} Q &= HW_Q \\ K &= HW_K \\ V &= HW_V. \end{aligned}$$

Mallin on määritettävä, kuinka paljon kukin tokeni huomioi muita tokeneita. Tätä varten kyselyiden ja avainten välille lasketaan pistetulot, joista muodostuu huomiopistematriisi

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

Matriisin S alkio S_{ij} kuvaa position i tokenin kyselyn ja position j tokenin avaimen välistä vastaavuutta. Nimittäjä $\sqrt{d_k}$ skaalaa pistetuloja, koska niiden varianssi

kasvaa tyypillisesti kysely- ja avainvektorien dimension mukana. Skaalaus ehkäisee softmax-funktion kyllästymistä ja auttaa ylläpitämään stabiilimpia gradientteja koulutuksen aikana [42].

Huomiopisteet muunnetaan riveittäin painoiksi softmax-funktion avulla:

$$A_{ij} = \frac{\exp(S_{ij})}{\sum_{r=1}^T \exp(S_{ir})}.$$

Tällöin $A_{ij} \geq 0$, ja jokaiselle i :lle pätee

$$\sum_{j=1}^T A_{ij} = 1.$$

Paino A_{ij} ilmaisee siten, kuinka suuri osuus position j arvovektorista sisällytetään position i uuteen esitykseen.

Itsehuomiomekanismin tulos muodostetaan arvovektoreiden painotettuna summana

$$\text{Attention}(Q, K, V) = AV.$$

Indeksissä i sijaitsevan tokenin kohdalla laskenta voidaan esittää muodossa

$$o_i = \sum_{j=1}^T A_{ij} v_j,$$

missä $v_j \in \mathbb{R}^{d_v}$ on arvomatriisin V riviä j vastaava arvovektori. Indeksien i tokenin uusi esitys o_i muodostuu siis kaikkien tokenien arvovektoreista, mutta kunkin arvovektorin vaikutus määräytyy tokenien välisten huomiopainojen perusteella.

5.2 Kausaalinen itsehuomio

Dekooderipohjaisessa Transformerissa huomio rajoitetaan kausaalisella maskilla siten, että kohdassa t malli voi hyödyntää vain tokeneita $w_1, \dots, w_{t-1}$. Tällöin malli säilyttää autoregressiivisen tulkinnan

$$P(w_t \mid w_1, \dots, w_{t-1}).$$

Autoregressiivisen kielimallin tavoitteena on ennustaa seuraava tokeni ainoastaan aiemmin havaitun kontekstin perusteella. Tämän vuoksi mallin ei saa antaa hyödyntää ennustettavaa tokenia eikä sen jälkeisiä tokeneita.

Määritellään maskimatriisi $M \in \mathbb{R}^{T \times T}$ siten, että

$$M_{ij} = \begin{cases} 0, & \text{jos } j \leq i, \\ -\infty, & \text{jos } j > i. \end{cases}$$

Tällöin kausaalinen itsehuomio määritellään muodossa

$$\text{CausalAttention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

Koska $\exp(-\infty) = 0$, positiota i vastaavan softmax-jakauman painot positioidille $j > i$ ovat nollija. Siten tokenin i esitys voi riippua vain tokeneista $x_1, \dots, x_i$.

Kausaalinen maski erottaa dekooderipohjaisen kielimallin esimerkiksi bidirektio-naalisesta Transformer-enkooderista. Enkooderipohjaisissa malleissa, kuten BERT-mallissa, tokenin esitys voi hyödyntää sekä vasemmalla että oikealla sijaitsevaa kontekstia [11]. Tämä soveltuu hyvin esimerkiksi tekstin luokitteluun ja maskattujen tokenien ennustamiseen, mutta ei suoraan vasemmalta oikealle etenevään tekstigenerointiin. Generatiivisissa suurissa kielimalleissa käytetään tämän vuoksi yleensä kausaalista itsehuomiota.

5.3 Monipäinen itsehuomio

Yksi huomio-operaatio muodostaa tokenien väliset riippuvuudet yhden kysely-, avain- ja arvoprojektion avulla. Vaswani ym. [42] esittivät monipäisen huomion (engl. *multi-head attention*), jossa malli laskee useita rinnakkaisia huomio-operaatioita erillisillä parametreilla. Olkoon huomioiden päiden lukumäärä h ja olkoon kunkin pään kysely- ja avainvektorien dimensio d_h . Pään r projektiot voidaan määritellä muodossa

$$\begin{aligned} Q^{(r)} &= HW_Q^{(r)}, \\ K^{(r)} &= HW_K^{(r)}, \\ V^{(r)} &= HW_V^{(r)}, \end{aligned}$$

missä

$$\begin{aligned} W_Q^{(r)}, W_K^{(r)} &\in \mathbb{R}^{d \times d_h}, \\ W_V^{(r)} &\in \mathbb{R}^{d \times d_v}. \end{aligned}$$

Kunkin pään tulos saadaan kausaalisen huomion avulla:

$$\text{head}^{(r)} = \text{CausalAttention}(Q^{(r)}, K^{(r)}, V^{(r)}).$$

Päiden tulokset ketjutetaan ja projisoidaan takaisin mallin piiloavaruuteen:

$$\text{MHA}(H) = \text{Concat}(\text{head}^{(1)}, \dots, \text{head}^{(h)}) W_O,$$

missä

$$W_O \in \mathbb{R}^{hd_v \times d}.$$

Monipäinen rakenne kasvattaa mallin kykyä muodostaa erilaisia kontekstiriippuvuuksia samanaikaisesti. Eri huomio-operaatiot voivat oppia painottamaan esimerkiksi lähellä olevia tokeneita, lauseen eri osien välisiä riippuvuuksia tai tekstin rakenteellisia merkkejä. On kuitenkin huomattava, ettei yksittäiselle huomiopäälle välttämättä voida antaa yksikäsitteistä kielellistä tulkintaa. Huomiopainot ovat mallin sisäisiä laskennallisia suureita, eivätkä ne sellaisenaan muodosta täydellistä selitystä mallin tuottamalle ennusteelle.

5.4 Transformer-dekooderikerros

Nykyaikainen dekooderipohjainen Transformer muodostuu useista peräkkäisistä Transformer-kerroksista. Jokainen kerros sisältää tavallisesti monipäisen kausaalisen itsehuomion, tokenikohtaisen eteenpäinsyöttävän neuroverkon, residuaaliyhteydet sekä normalisointikerrokset.

Olkoon $H^{(\ell-1)} \in \mathbb{R}^{T \times d}$ kerroksen $\ell-1$ tuottama tokeniesitysmatriisi. Esimerkiksi esinormalisoidussa (engl. *pre-normalization*) Transformer-kerroksessa itsehuomioali-kerroksen tulos voidaan kirjoittaa muodossa

$$\widetilde{H}^{(\ell)} = H^{(\ell-1)} + \text{MHA}^{(\ell)} \left(\text{Norm}_1^{(\ell)} \left(H^{(\ell-1)} \right) \right).$$

Tämän jälkeen sovelletaan tokenikohtaista eteenpäinsyöttävää verkkoa:

$$H^{(\ell)} = \widetilde{H}^{(\ell)} + \text{FFN}^{(\ell)} \left(\text{Norm}_2^{(\ell)} \left(\widetilde{H}^{(\ell)} \right) \right).$$

Residuaaliyhteys tarkoittaa, että alikerroksen syöte lisätään sen tulokseen. Tällä on käytännössä suuri merkitys syvien verkkojen optimoinnille, koska residuaaliyhteydet helpottavat gradientin kulkua useiden kerrosten läpi. Tämä liittyy luvussa 4 käsiteltyihin gradienttipohjaisen optimoinnin haasteisiin.

Alkuperäisessä Transformerissa käytettiin kerrosnormalisointia (engl. *Layer Normalization*) [?]. Tokenivektorille $h \in \mathbb{R}^d$ se voidaan määritellä muodossa

$$\text{LayerNorm}(h) = \gamma \odot \frac{h - \mu(h)\mathbf{1}}{\sqrt{\sigma^2(h) + \varepsilon}} + \beta,$$

missä

$$\begin{aligned} \mu(h) &= \frac{1}{d} \sum_{i=1}^d h_i, \\ \sigma^2(h) &= \frac{1}{d} \sum_{i=1}^d (h_i - \mu(h))^2. \end{aligned}$$

Vektorit $\gamma, \beta \in \mathbb{R}^d$ ovat opittavia parametreja, $\odot$ tarkoittaa komponenttikohtaista kertolaskua ja $\varepsilon > 0$ on numeerista vakautta parantava vakio. Useissa nykyaikaisissa kielimalleissa käytetään LayerNormin sijasta esimerkiksi RMSNorm-normalisointia [?], mutta normalisoinnin yleinen tehtävä on sama: se tukee mallin vakaata optimointia.

Transformer-kerroksen toinen keskeinen osa on eteenpäinsyöttävä verkko (engl. *feed-forward network*, FFN). Toisin kuin itsehuomio, joka yhdistää informaatiota eri tokenipositioiden välillä, FFN toimii erikseen jokaisessa tokenipositiossa. Perusmuodossaan se voidaan esittää muodossa

$$\text{FFN}(h) = W_2 \phi(W_1 h + b_1) + b_2,$$

missä

$$\begin{aligned} W_1 &\in \mathbb{R}^{d_{\text{ff}} \times d}, \\ W_2 &\in \mathbb{R}^{d \times d_{\text{ff}}}, \end{aligned}$$

ja d_{ff} on yleensä mallin piilodimensiota d suurempi väliavaruuden dimensio. Aktiivaatiofunktio ϕ voi olla esimerkiksi ReLU-, GELU- tai SiLU-funktio. Useissa nykyaikaisissa LLM-arkkitehtuureissa käytetään myös portitettua FFN-rakennetta, kuten SwiGLU-muotoa [?].

Kun Transformer-kerroksia on L , mallin lopullinen esitysmatriisi on

$$H^{(L)} \in \mathbb{R}^{T \times d}.$$

Sen rivi $h_t^{(L)}$ sisältää tokeniposition t kontekstualisoidun esityksen. Kausaalisuuden vuoksi tämä esitys voi riippua ainoastaan tokeneista $x_1, \dots, x_t$.

5.5 Seuraavan tokenin ennustaminen ja kielimallin tavoitefunktio

Kuten luvussa 2.2 esitetyn n -gram-mallin ja edellisessä luvussa käsitellyn MLP-kielimallin tapauksessa, myös Transformer-pohjaisen kielimallin tavoitteena on muodostaa ehdollinen todennäköisyysjakauma seuraavalle tokenille. Erona on se, että Transformer muodostaa kontekstin esityksen useiden itsehuomio- ja eteenpäinsyötävien kerrosten avulla.

Olkoon $h_t^{(L)} \in \mathbb{R}^d$ position t viimeisen Transformer-kerroksen esitys. Sanaston tokenien normalisoimattomat pistemäärät eli logitit määritellään muodossa

$$z_t = W_{\text{vocab}} h_t^{(L)} + b_{\text{vocab}},$$

missä

$$\begin{aligned} W_{\text{vocab}} &\in \mathbb{R}^{|V| \times d}, \\ b_{\text{vocab}} &\in \mathbb{R}^{|V|}. \end{aligned}$$

Logitvektorin z_t komponentti $z_{t,j}$ vastaa tokenin v_j pistemäärää. Seuraavan tokenin ehdollinen todennäköisyysjakauma saadaan softmax-funktiolla:

$$p_{\theta}(x_{t+1} = v_j \mid x_1, \dots, x_t) = \frac{\exp(z_{t,j})}{\sum_{k=1}^{|V|} \exp(z_{t,k})}.$$

Koko tokenisekvenssin todennäköisyys voidaan kirjoittaa autoregressiivisena tulona:

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^{T-1} p_{\theta}(x_{t+1} \mid x_1, \dots, x_t).$$

Käytännössä ensimmäisen tokenin ennustaminen voidaan määritellä aloitustokenin avulla. Jos $x_0 = \langle s \rangle$, voidaan sama esitys kirjoittaa muodossa

$$p_{\theta}(x_1, \dots, x_T \mid x_0) = \prod_{t=0}^{T-1} p_{\theta}(x_{t+1} \mid x_0, \dots, x_t).$$

Tämä on suora yleistys luvussa 2.2 esitetystä Markov-oletukseen perustuvasta kielimallista. n -gram-mallissa konteksti rajoitetaan eksplisiittisesti viimeiseen $n - 1$

tokeniin, kun taas Transformer-mallissa ehdollinen jakauma voi riippua kaikista käytettävissä olevista aiemmista tokeneista. Käytännössä riippuvuutta rajoittaa mallin suurin sallittu kontekstipituus.

Mallin parametrit θ estimoidaan minimoimalla negatiivinen log-uskottavuus eli tokenitason ristientropiahäviö:

$$\mathcal{L}(\theta; X) = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_1, \dots, x_t).$$

Sekvenssin pituudella normalisoitu häviö on

$$\overline{\mathcal{L}}(\theta; X) = - \frac{1}{T-1} \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_1, \dots, x_t).$$

Tavoitefunktio on siten periaatteellisesti sama kuin aiemmin käsitellyissä maksimiuskottavuuteen perustuvissa kielimalleissa. Keskeinen ero on mallin parametrisoinnissa: Transformer ei talleta erillistä jakaumaa jokaiselle diskreetille kontekstille, vaan muodostaa ehdollisen jakauman syvän, jaettuihin parametreihin perustuvan funktion avulla.

Koulutusvaiheessa koko syötesekvenssi voidaan käsitellä samanaikaisesti. Kausaalinen maski varmistaa, ettei tokenia x_{t+1} käytetä sitä ennustavan position t laskennassa. Tämän ansiosta kaikkien positioiden ristientropiahäviöt voidaan laskea rinnakkain, vaikka mallin määrittelemä todennäköisyysjakauma on autoregressiivinen. Parametrien gradientit saadaan vastavirta-algoritmillä, ja parametreja päivitetään luvussa 4 esitetyillä gradienttipohjaisilla menetelmillä. Käytännössä suurten kielimallien esikoulutuksessa käytetään usein AdamW-optimointimenetelmää.

Tokenitason keskimääräisestä negatiivisesta log-uskottavuudesta voidaan määrittellä myös perplexiteetti:

$$\text{PPL} = \exp(\overline{\mathcal{L}}(\theta; X)).$$

Perplexiteetti mittaa mallin keskimääräistä epävarmuutta havaittuja tokeneita kohtaan. Pieni perplexiteetti tarkoittaa, että malli antaa aineiston tokeneille keskimäärin suuren todennäköisyyden. Mittari ei kuitenkaan sellaisenaan arvioi esimerkiksi vastauksen faktuaalista oikeellisuutta, turvallisuutta, hyödyllisyyttä tai kykyä hyödyntää ulkoisia lähteitä.

5.6 Autoregressiivinen generointi

Koulutuksen jälkeen mallia käytetään tekstin generointiin iteratiivisesti. Olkoon alkuperäinen syötekonteksti

$$(x_1, \dots, x_t).$$

Malli muodostaa ensin jakauman

$$p_{\theta}(x_{t+1} \mid x_1, \dots, x_t).$$

Tämän jälkeen jakaumasta valitaan uusi token $\hat{x}_{t+1}$, joka liitetään kontekstiin. Seuraavalla askeleella malli muodostaa jakauman

$$p_{\theta}(x_{t+2} \mid x_1, \dots, x_t, \hat{x}_{t+1}).$$

Prosessia jatketaan, kunnes generoidaan lopetustokeni tai saavutetaan ennalta määritetty enimmäispituus.

Yksinkertaisin valintasääntö on ahne dekodaus, jossa valitaan aina todennäköisin tokeni:

$$\hat{x}_{t+1} = \arg \max_{v_j \in V} p_{\theta}(x_{t+1} = v_j \mid x_1, \dots, x_t).$$

Vaihtoehtoisesti tokeni voidaan valita satunnaisotannalla mallin ennustamasta jakaumasta. Tällöin generoinnin satunnaisuutta voidaan säätää lämpötilaparametrilla $\tau > 0$:

$$p_{\theta, \tau}(x_{t+1} = v_j \mid x_1, \dots, x_t) = \frac{\exp(z_{t,j}/\tau)}{\sum_{k=1}^{|V|} \exp(z_{t,k}/\tau)}.$$

Kun $\tau < 1$, jakauma keskittyy suuremman todennäköisyyden tokeneihin. Kun $\tau > 1$, jakauma tasoittuu ja generointi muuttuu satunnaisemmaksi. Käytännössä voidaan käyttää myös esimerkiksi top- k - ja top- p -näytteenottoa, joissa näytteenotto rajoitetaan todennäköisimmiksi arvioituihin tokeneihin.

Autoregressiivinen generointi poikkeaa koulutuksesta laskennallisesti siinä, että uusi token lisätään kontekstiin yksi kerrallaan. Tehokkuuden parantamiseksi toteutuksissa talletetaan aiemmissa generointivaiheissa lasketut avain- ja arvovektorit niin sanottuun avain–arvovälimuistiin (engl. *key-value cache*). Tällöin aikaisempien tokenien avain- ja arvoprojektioita ei tarvitse laskea uudelleen jokaisella generointiaskeleella.

5.7 Skaalaus ja jälkikoulutus

Suurten kielimallien suorituskykyyn vaikuttavat muun muassa parametrien määrä, harjoitusaineiston määrä ja laatu, koulutuksessa käytettyjen tokenien määrä, käytettävissä oleva laskentakapasiteetti sekä konteksti-ikkunan pituus. Skaalauslakeja koskevissa tutkimuksissa on havaittu, että kielimallien ennustehäviö yleensä pienee säännönmukaisesti mallin, aineiston ja laskennan määrän kasvaessa [?, ?]. Pelkkä parametrimäärä ei kuitenkaan yksin määritä mallin laatua, sillä myös koulutusaineiston koostumus, optimointi, arkkitehtuurivalinnat ja jälkikoulutus vaikuttavat olennaisesti mallin toimintaan.

Esikoulutus tarkoittaa tavallisesti edellä määritellyn seuraavan-tokenin-ennustustehtävän optimointia laajalla tekstiaineistolla. Esikoulutettu malli oppii tekstin tilastollisia, syntaktisia ja semanttisia säännönmukaisuuksia, mutta sen käyttäytyminen ei vielä välttämättä vastaa vuorovaikutteisen avustajan käyttöä. Tämän vuoksi esikoulutusta seuraa usein jälkikoulutus, jossa mallia sovitetaan ohjeita ja keskustelumutoista vuorovaikutusta sisältävään aineistoon.

Ohjatulla hienosäädöllä (engl. *supervised fine-tuning*) malli optimoidaan esimerkiksi käyttäjän ohjeen ja tavoitellun vastauksen muodostamalla pareilla. Lisäksi mallin tuottamia vastauksia voidaan optimoida ihmisten tai muiden arvioijien ilmaisten preferenssien perusteella. Yksi tunnettu lähestymistapa on ihmispalautteeseen perustuva vahvistusoppiminen (engl. *reinforcement learning from human feedback*, RLHF) [?]. Jälkikoulutus ei muuta sitä perustavaa tosiasiaa, että malli tuottaa tekstin autoregressiivisesti seuraavaa tokenia ennustamalla. Se kuitenkin vaikuttaa merkittävästi siihen, millaisia vastauksia malli todennäköisesti tuottaa käyttäjän antamissa tilanteissa.

5.8 earlier subsection that should come later

Olkoon V sanasto, $X = (x_1, x_2, \dots, x_T)$ syötesekvenssi one-hot vektoreita, T sekvenssin pituus, ja $E \in \mathbb{R}^{d \times |V|}$ upotusmatriisi, jolloin Ex_t on syötesekvenssin t :nnen tokenin upotusvektori.

Itsehuomiomekanismi ei sellaisenaan sisällä tietoa tokenien järjestyksestä. Jos syötevektorien järjestystä vaihdetaan, pelkkään itsehuomioon perustuva laskenta tuottaa vastaavasti vain permutoidun tuloksen. Tämän vuoksi tokenien esityksiin on liitettävä sijaintia kuvaava informaatio. Alkuperäisessä Transformerissa käytettiin deterministisiä sinimuotoisia positiokoodauksia [42]. Tällöin tokenin ensimmäisen kerroksen esitys voidaan kirjoittaa muodossa

$$h_t^{(0)} = e_t + p_t,$$

missä $p_t \in \mathbb{R}^d$ on tokenin t positiovektori.

Alkuperäisen Transformer-mallin sinimuotoisessa positiokoodauksessa komponentit määritellään esimerkiksi seuraavasti:

$$\begin{aligned} p_{t,2i} &= \sin\left(\frac{t}{10000^{2i/d}}\right), \\ p_{t,2i+1} &= \cos\left(\frac{t}{10000^{2i/d}}\right), \end{aligned}$$

missä i on vektorin komponentti-indeksi. Nykyaikaisissa suurissa kielimalleissa käytetään usein muita sijaintimenetelmiä, kuten opittuja positioupotuksia tai rotaatiopohjaisia positioupotuksia (engl. *rotary positional embeddings*, RoPE) [?]. Näiden yksityiskohdat vaihtelevat mallikohtaisesti, mutta niiden yhteinen tarkoitus on antaa mallille tieto tokenien järjestyksestä ja etäisyyksistä.

5.9 Parametrinen kielimallin rajoitteet ja yhteys RAG-järjestelmiin

Transformer-pohjainen suuri kielimalli voi tuottaa sujuvaa ja kontekstuaalisesti johdonmukaista tekstiä, mutta sen tietämys ei ole tietokantamuodossa eikä sen parametreihin tallentunut informaatio ole välttämättä ajantasaista, täydellistä tai helposti todennettavaa. Malli voi myös tuottaa uskottavan tuntuista väitteitä, jotka eivät perustu luotettavaan lähteeseen tai ovat virheellisiä. Tätä ilmiötä kutsutaan usein hallusinoinniksi, vaikka kyse ei ole mallin sisäisestä havaintokokemuksesta, vaan todennäköisyyspohjaisen generointiprosessin tuottamasta virheellisestä tai perusteettomasta tekstistä.

Lisäksi mallin käytettävissä oleva konteksti on rajallinen, ja mallin kyky hyödyntää sille annettua kontekstia riippuu sekä syötteen rakenteesta että mallin koulutuksesta. Se, että asiakirja sisällytetään mallin kontekstiin, ei takaa, että malli käyttää asiakirjaa oikein tai että sen tuottama vastaus perustuu yksinomaan kyseiseen asiakirjaan.

Hakua hyödyntävä generointi (engl. *retrieval-augmented generation*, RAG) pyrkii lieventämään näitä rajoitteita liittämällä mallin generointikontekstiin ulkoisesta tietolähteestä haettuja dokumentteja [?]. Tällöin kielimallin parametreihin sisältyvää tietoa täydennetään generointihetkellä saatavilla olevalla dokumenttipohjaisella tiedolla. Menetelmä voi parantaa erityisesti ajantasaisen, organisaatio- tai toimialakohtaisen ja lähteistettävän tiedon hyödyntämistä. Se tuo kuitenkin mukanaan uusia mahdollisia virhelähteitä, kuten epäonnistuneen haun, epärelevanttien dokumenttien sisällyttämisen kontekstiin sekä dokumenttien virheellisen tulkinnan.

Seuraavissa luvuissa tarkastellaan retrieval-augmented generation -järjestelmän toteutusta ja arvioidaan, miten Transformer-pohjainen suuri kielimalli toimii käytännön sovelluksessa, jossa generointia tuetaan ulkoisesta tietolähteestä haetulla kontekstilla.

6 Metodologia

6.1 Tutkimusasetelma

Tutkimus toteutetaan empiirisenä tapaustutkimuksena, jossa arvioidaan yhden kielimallipohjaisen järjestelmän toimintaa rajatussa sovelluskontekstissa. Tavoitteena ei ole rakentaa uutta mallia tai vertailla useita eri järjestelmiä keskenään, vaan tarkastella, miten Cortex Analyst suoriutuu käytännöllisestä tehtävästä, jossa luonnollisen kielen kysymykset muunnetaan SQL-kyselyiksi. Tapaustutkimus soveltuu tähän tarkoitukseen hyvin, koska se mahdollistaa järjestelmän toiminnan yksityiskohtaisen tarkastelun todellisessa kaupallisessa ympäristössä.

Tutkimuksen analyysiyksikkönä on yksittäinen käyttäjän luonnollisen kielen kysymys ja sitä vastaava Cortex Analystin tuottama SQL-kysely. Arviointi kohdistuu siihen, vastaako muodostettu kysely käyttäjän tarkoitusta, hyödyntääkö se oikeita tauluja ja kenttiä sekä noudattaako se tietokannan rakennetta. Näin tutkimus keskittyy suoraan siihen vaiheeseen, jossa kielimallin tulkinta muuttuu koneellisesti suoritettavaksi tietokantakyselyksi.

6.2 Tutkimusaineisto

Tutkimuksen aineistona toimii puhelinkeskusdataa sisältävä tietokanta, joka noudattaa kuvassa 1 esitettyä rakennetta. Tarkastelun keskiössä on yksi keskusteludataa sisältävä päätason taulu, jota täydentävät kolme liitännäistaulua. Liitännäistaulujen avulla voidaan tarkentaa päätason taulun yksittäisten kenttien sisältöä ja merkitystä. Tietokannan rakenne on siten riittävän monipuolinen, jotta voidaan tarkastella sekä yksinkertaisia että vaativampia kyselytilanteita, kuten taululiitoksia, suodatuksia ja aggregaatioita.

Aineisto sisältää kaupallisesti arkaluonteista tietoa, minkä vuoksi varsinaisia kyselytuloksia ei julkaista. Tästä syystä tutkimuksessa vertaillaan järjestelmän tuottamia SQL-kyselyitä eikä niiden palauttamia tietosisältöjä. Ratkaisu tukee myös analyysin johdonmukaisuutta, koska virheellisen kyselyn vaikutus lopputulokseen ei ole aina pääteltävissä pelkästään syntaksista, vaan riippuu osittain tietokannan sisällöstä.

6.3 Cortex Analyst ja semanttinen näkymä

Tutkimuksessa käytettävä järjestelmä on Snowflaken Cortex Analyst [39]. Järjestelmän tarkoituksena on mahdollistaa tietokantatiedon hyödyntäminen luonnollisen kielen kautta siten, että käyttäjän ei tarvitse tuntea SQL-kieltä tai tietokannan skeemaa yksityiskohtaisesti. Cortex Analyst muodostaa käyttäjän kysymyksestä SQL-kyselyn ja voi suorittaa sen tietokantaa vasten.

Cortex Analystin toiminta perustuu semanttiseen näkymään, jossa tietokannan taulut, kentät ja niiden merkitykset kuvataan YAML-muotoisena määrittelynä (ks. kuva 2). Semanttinen näkymä tarjoaa kielimallille rakenteellisen kuvauksen siitä, mitä tietoa tietokannassa on saatavilla ja miten eri taulut liittyvät toisiinsa. Näin järjestelmä pystyy muodostamaan syntaktisesti valideja SQL-kyselyitä ja kohdistamaan kyselyn oikeisiin tietokannan osiin. Semanttisen näkymän laatu on keskeinen

osa järjestelmän toimivuutta, koska kielimallin tulkinta perustuu olennaisesti siihen, miten skeema on kuvattu.

Tutkimuksessa käytetty semanttinen näkymä on tutkijan rakentama yhteistyössä yksityisen IT-konsulttiyrityksen Knowitin ja yhden heidän asiakkaan kanssa osana hanketta, jossa pyritään hyödyntämään kielimalleja helpottamaan tietokannan käyttöä.

Tässä tutkimuksessa käytetty semanttinen näkymä kuvailee kaikki kuvassa 1 esiintyvät taulut, niiden kentät ja suhteet toisiinsa sekä useita kentistä laskettuja metriikoita. Kuvassa 2 on esitetty yksinkertaistettu versio käytetystä semanttisesta näkymästä.

```
name: SEM_EXAMPLE
description: |-
  Semantic model that models conversations data from Genesys
  Cloud contact center software.
tables:
  - name: CONV
    description: |-
      All Genesys conversations.
    base_table:
      database: DEV
      schema: DBT_OM
      table: FACT_GENESYS__CONVERSATIONS_STANDARD
    dimensions:
      - name: C_ADDRESS_FROM
        description: Source address for the interaction. Only
          present in email contacts..
        expr: conv.address_from
        data_type: VARCHAR(16777216)
    facts:
      - name: CALL_DURATION
        description: Duration of the actual call in seconds.
          The time the customer speaks to an agent
        expr: conv.call_duration
        data_type: NUMBER(38,0)
        access_modifier: public_access
    metrics:
      - name: AVERAGE_CALL_DURATION
        description: Average call duration in seconds
        expr: sum(conv.call_duration)
        access_modifier: public_access
    primary_key:
      columns:
        - PARTICIPANTS_ID
        - IDENTIFIER
```

Kuva 2: Esimerkki semanttisesta näkymästä. Esimerkki on yksinkertaistettu versio tutkimuksessa käytetystä näkymästä.

Kokeiden aikana Cortex Analyst käytti taustallaan Anthropicin Claude Sonnet 4 -mallia [1]. Tutkimuksen tulokset kuvaavat ensisijaisesti juuri tämän järjestelmän ja mallin yhdistelmän toimintaa. Huhtikuussa 2026 Cortex Analyst voi käyttää uudempaa Claude Sonnet 4.6 -mallia [39]. Tuloksia ei siksi voida sellaisenaan yleistää kaikkiin kielimalleihin tai muihin luonnollisen kielen tietokantarajapintoihin.

6.4 Arviointiperusteet

Tuotettuja SQL-kyselyitä arvioidaan useiden toisiaan täydentävien kriteerien avulla. Ensinnäkin tarkastellaan, valitseeko järjestelmä oikeat taulut ja käyttääkö se oikeita liitoksia silloin, kun käyttäjän kysymys edellyttää useamman taulun yhdistämistä. Toiseksi arvioidaan suodatusehtoja, erityisesti where-lauseiden sisältöä, koska pienikin virhe niissä voi muuttaa kyselyn merkitystä olennaisesti. Lisäksi huomioidaan aggregaatiot, duplikaattien käsittely ja sarakevalinnat.

Syntaktista validiutta ei tarkastella erikseen, koska kaikki järjestelmän tuottamat kyselyt olivat syntaktisesti valideja.

Arvioinnissa erotetaan toisistaan neljä poikkeamatyyppiä. Kysely luokitellaan korrektiksi, jos sen perusteella saatava vastaus vastaisi esitettyyn luonnollisen kielen kysymykseen oikein ja huomioisi ne implisiittiset taustaoletukset, jotka ovat sovellusalueen näkökulmasta perusteltuja.

Tutkittava tietokanta sisältää puhelinkeskusdataa, joten erityisesti puhelun suuntaan liittyvät taustaoletukset ovat keskeisiä. Esimerkiksi palvelutasoa koskevan kysymyksen implisiittisenä taustaoletuksena pidetään sitä, että tarkastelu kohdistuu vain saapuviin kontakteihin. Korrekti kysely huomioi tällaiset taustaoletukset ja vastaa käyttäjän tiedontarpeeseen oikein.

Kysely merkitään melkein korrektiksi, jos generoitu SQL vastaa olennaisilta osin kysymykseen, mutta sisältää teknisen puutteen. Esimerkiksi kysely, josta puuttuu NULLIF -lause jakolaskun nimittäjästä, vastaa sisällöllisesti samaan kysymykseen kuin NULLIF-avainsanan sisältävä kysely, mutta voi erikoistapauksessa johtaa virheeseen. Tällainen kysely luokitellaan melkein korrektiksi.

Kysely luokitellaan virheelliseksi, jos se ei vastaa kysymykseen merkityksellisellä tavalla. Esimerkiksi kysely, joka käyttää väärää suodatusta tai josta puuttuu olennainen taulu tai kenttä, merkitään virheelliseksi.

Jos malli ei tuota lainkaan SQL-kyselyä, vaan ilmoittaa, että kysymykseen ei voida vastata, tämä luokitellaan erikseen ei kyselyä -kategoriaksi. Kaikkiin tutkimuksessa käytettyihin kysymyksiin on mahdollista vastata käytettävissä olevan aineiston ja semanttisen näkymän perusteella, joten tähän kategoriaan kuuluvat vastaukset ovat virheellisiä. Ne on kuitenkin mielekästä erottaa muista virheellisistä vastauksista, koska ne eivät tuota harhaanjohtavia tuloksia.

Samaan luonnollisen kielen kysymykseen voi olla useita oikeellisia SQL-vastauksia. Esimerkiksi kysymykseen How many contacts did we handle in January 2026? järjestelmä voi tuottaa kyselyn, joka palauttaa yhden kokonaisluvun, tai kyselyn, joka palauttaa taulun sisältäen kontaktien määrän tammi-, helmi- ja maaliskuulle, joista käyttäjä voi poimia vastauksen. Tässä tutkielmassa kysely tulkitaan oikeelliseksi siinä tapauksessa, että sen tuottama vastaus sisältää käyttäjän tiedontarpeen.

Tästä johtuen oikeellisuuden lisäksi tutkimuksen keskeinen kiinnostuksen kohde on kielimallin tuottamien kyselyiden vaihtelu. Tämän vuoksi analysoidaan myös, kuinka monta uniikkia SQL-kyselyä järjestelmä tuottaa kutakin kysymystä kohden, kun sama luonnollisen kielen kysymys tai jokin sen variaatioista esitetään useita kertoja.

Kyselyä ei pidetä uniikkina, jos semanttisesti ekvivalentti kysely on tuotettu aiemmin, paitsi siinä tapauksessa, että toinen kysely hyödyntää semanttisessa näkymässä valmiiksi määriteltä metriikkaa ja toinen tuottaa laskukaavan, joka on ekvivalentti semanttisessa näkymässä määritellyn metriikan kanssa. Esimerkiksi jos semanttiseen näkymään ollaan määriteltä metriikka `total_calls` kaavalla `select sum(*) from conv`, niin kyselyt `select sum(*) from conv` ja `select total_calls from semantic_view` nähdään toisistaan uniikkeina.

Päätös perustellaan sillä, että nämä kaksi lähestymistapaa ovat olennaisesti erilaiset (käytettävissä olevan kontekstin soveltaminen vs. päättely). Vaikka kyselyt ovat sisällöllisesti semanttisesti samankaltaisia, niiden muodostaminen edellyttää erilaista ongelmanratkaisua ja hyödyntää järjestelmältä eri kyvykkyyksiä. Ensimmäisessä tapauksessa malli tunnistaa valmiiksi määritellyn metriikan ja osaa käyttää sitä suoraan, kun taas jälkimmäisessä tapauksessa sen on pääteltävä tarvittava laskentatapa. Tämän vuoksi tapaukset erotellaan analyysissa toisistaan, vaikka niiden semanttinen tavoite on sama.

Tuotettujen SQL-kyselyiden oikeellisuus arvioitiin manuaalisesti vertaamalla niitä tutkijan tulkintaan siitä, millainen kysely vastaisi käyttäjän tiedontarvetta. Arvioinnin suoritti yksi arvioija, mikä heikentää luokittelun reliabiliteettia.

7 Kokeet

Ensimmäisessä kokeessa tarkastellaan Cortex Analystin konsistenssia, eli sitä, tuottaako järjestelmä saman SQL-kyselyn, kun sama luonnollisen kielen kysymys esitetään useita kertoja.

Toisessa kokeessa tarkastellaan järjestelmän herkkyyttä sellaisille syötteen muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä. Näissä kokeissa muodostetaan useita eri sanamuotoja samasta tiedontarpeesta ja arvioidaan, tuottaako järjestelmä niistä olennaisesti saman SQL-kyselyn.

Konsistenssikoe Koska kielimallit ovat tilastollisia järjestelmiä, sama syöte ei välttämättä aina johda täsmälleen samaan tulosteeseen. Tietokantakyselyiden tapauksessa tämä on olennainen kysymys, koska käyttäjät odottavat järjestelmältä ennustettavaa ja toistettavaa toimintaa.

Konsistenssikokeessa sama luonnollisen kielen kysymys esitetään järjestelmälle 10 kertaa. Jokaisesta suorituksesta tallennetaan järjestelmän tuottama SQL-kysely, joita verrataan keskenään. Käytetyt kysymykset ovat seuraavat:

- A. *How many total contacts did we handle in January 2026?*
- B. *What's our answer rate for calls in February 2026?*
- C. *What percentage of contacts that were answered within our service level target were closed by the customer versus closed by the agent?*
- D. *What was our peak contact hours by day of week? Show me the distribution of when contacts arrive throughout the day, broken down by hour and weekday.*
- E. *What's the success rate of our callbacks? Show me how many callbacks resulted in a conversation versus no answer, and what the average time between the original inbound contact and the callback was.*

Määrittelyssä semanttisessa näkymässä on kuvattu metriikoihin SQL-kyselyt, jotka vastaavat suoraan kysymyksiin A ja B. Kysymykset C-E ovat huomattavasti monimutkaisempia, ja niihin ei ole ennalta validoituja SQL-kyselyitä semanttisessa näkymässä. Kysymys D vaatii, että kentästä `CONTACT_CONNECTED_DATETIME` erotetaan kellonaika tunnin tarkkuudella, ja viikonpäivä pitää hakea kalenteritaulusta. Kysymys E on rakenteeltaan monitulkintainen. Erityisesti takaisinsoiton ja alkuperäisen sisääntulevan kontaktin välisen ajan keskiarvoa koskeva osa voidaan tulkita vaativan kahden eri rivin välistä vertailua, vaikka käytetyssä aineistossa vastaus voidaan muodostaa yhdeltä takaisinsoittoa kuvaavalta riviltä.

Sensitiivisyyskoe Kokeen tavoitteena on arvioida, tulkitseeko Cortex Analyst semanttisesti saman sisältöiset kysymykset johdonmukaisesti vai johtavatko kielelliset erot erilaisiin SQL-kyselyihin. Jos järjestelmä on hyvin herkkä muotoilun vaihtelulle, luonnollisen kielen käyttö rajapintana muuttuu käyttäjän kannalta epävarmaksi. Jos taas järjestelmä tuottaa olennaisesti saman kyselyn eri sanamuodoista huolimatta, tämä tukee sen käytettävyyttä käytännön työssä.

Koe toteutetaan muodostamalla kustakin konsistenssikokeessa käytetystä kysymyksestä 4 vaihtoehtoisia versiota, joissa pyritään säilyttämään kysymyksen merkitys, mutta muokkaamaan sanamuotoa. Järjestelmän tuottamat SQL-kyselyt tallennetaan ja vertaillaan keskenään. Käytetyt kysymykset ovat seuraavat:

A. Variaatioita kysymyksestä A:

1. *What was the total number of contacts we handled in January 2026?*
2. *How many contacts did we handle overall in January 2026?*
3. *What is the total contact volume for January 2026?*
4. *How many total customer contacts were handled during January 2026?*

B. Variaatioita kysymyksestä B:

1. *What was our call answer rate in February 2026?*
2. *What percentage of incoming calls were answered in February 2026?*
3. *What percentage of calls were answered in February 2026?*
4. *How did our call answer rate perform during February 2026?*

C. Variaatioita kysymyksestä C:

1. *Of the contacts answered within our service-level target, what share were closed by the customer compared to those closed by the agent?*
2. *For interactions resolved within the service level target, what percentage were customer-closed versus agent-closed?*
3. *Within our service level target, how is the closure split: what percent of contacts were closed by customers versus agents?*
4. *Among contacts answered within SLA, what proportion ended with customer closure versus agent closure?*

D. Variaatioita kysymyksestä D:

1. *When do contacts most frequently arrive during the week? Break this down by weekday and hour.*
2. *Show the hourly contact-arrival distribution for every day of the week, highlighting which hours were highest on each weekday.*
3. *What times during the day (hour-by-hour) drive the most contacts, split by weekday. Please summarize peak hours and the overall distribution.*
4. *Across the week, when do contacts arrive most often? Provide an hour-by-hour breakdown for each weekday and identify the peak contact hours.*

E. Variaatioita kysymyksestä E:

1. *What percentage of our callback attempts lead to an actual conversation? Please break down callbacks that reached a conversation vs. those with no answer, and calculate the average delay from the original inbound to the callback.*

2. *How effective are our callbacks? Show the success rate by counting callbacks that resulted in a conversation versus no answer, and report the average time-to-callback from the initial inbound contact.*
3. *For our callback campaign, what's the outcomes split and average response speed? Provide counts of conversations vs. no-answer callbacks, the resulting success rate, and the average elapsed time between inbound contact and callback.*
4. *Can you analyze callback performance for me: success rate (conversation vs no answer) with the number of callbacks in each outcome category, plus the average time lag from the inbound contact to when the callback occurred?*

Jokainen kysymys molemmissa kokeissa esitetään järjestelmälle 10 kertaa ja jokainen yksittäinen suoritus tehdään tyhjällä sessiolla. Testit suoritettiin 23.03.2026-28.03.2026 välisenä aikana käyttäen samaa semanttista mallia.

Kymmenen toistoa valittiin käytännöllisenä kompromissina, joka mahdollistaa vaihtelun havaitsemisen ilman kohtuuttoman suurta manuaalisen arvioinnin työmäärää.

8 Tulokset

Merkitään konsistenssikokeen kysymykset A-E ja herkkyysskokeen variaatiot A1-A4, B1-B4, C1-C4, D1-D4 ja E1-E4 listojen 7 ja 7 mukaisesti. Kokeiden tulokset on esitetty taulukoissa 2 ja 3. Taulukot kuvaavat kunkin kysymyksen osalta, kuinka monta kertaa järjestelmän tuottama SQL-kysely oli korrekti, melkein korrekti, virheellinen tai johti tilanteeseen, jossa SQL-kyselyä ei tuotettu lainkaan.

Konsistenssikokeessa vain kysymys C tuotti sekä korrekteja että virheellisiä SQL-kyselyitä. Kysymys B tuotti kaksi uniikkia SQL-kyselyä, joista toinen käytti semanttisessa näkymässä määriteltä metriikkaa `answer_percentage` suoraan ja toinen laski vastaavan prosenttiluvun erillisellä laskukaavalla.

Kysymykset A, D ja E tuottivat konsistenssikokeessa vain yhden uniikin lopputuloksen kymmenessä suorituksessa. Vaikka kysymys E tuotti jokaisella suorituskerralla virheellisen vastauksen siinä mielessä, että SQL-kyselyä ei muodostettu, konsistenssin näkökulmasta järjestelmä oli johdonmukainen, koska tuloste oli joka kerta sama. Kysymys E osoittaa, että konsistenssi ei yksin riitä kuvaamaan järjestelmän luotettavuutta, mikä heijastaa Text-to-SQL-evaluoinnissa tunnettua eroa toistettavuuden (konsistenssin) ja semanttisen virheettömyyden välillä [26].

Tulokset viittaavat siihen, että tässä tutkimusasetelmassa havaitut ongelmat olivat usein toistettavia, mikä on hyödyllinen havainto sekä käyttäjän että semanttisen näkymän kehittäjän näkökulmasta. Kysymys C kuitenkin osoittaa, että järjestelmä pystyi tuottamaan samassa asetelmassa sekä korrekteja että virheellisiä vastauksia, joten toistettavuus ei ollut täydellistä.

Alla on esimerkki kysymykseen E tuotetusta ei kyselyä -vastauksesta:

I'm sorry, but this question cannot be fully answered with the available data. While we can determine how many callbacks were answered versus not answered (using the `c_answered` field filtered for callback media type), the semantic model does not contain information about the original inbound contact that triggered a callback. There is no field linking a callback conversation back to its originating inbound contact, so we cannot calculate the time between the original inbound contact and the callback attempt.

Kysymykseen B järjestelmä tuotti kaksi uniikkia SQL-kyselyä. Näistä toinen käytti semanttisessa näkymässä ennalta määriteltä metriikkaa `answer_percentage` ja toinen muodosti vastaavan laskukaavan eksplisiittisesti SQL:ssä. Kysymykseen C järjestelmä tuotti kaksi uniikkia SQL-kyselyä, joista toinen oli korrekti ja toinen virheellinen. Korrekti kysely tuotettiin seitsemän kertaa ja virheellinen kolme kertaa. Virheellisessä kyselyssä järjestelmä sekoitti asiakkaan ja agentin roolit keskenään.

Konsistenssikokeen 50 suorituksesta 37 tuotti korrektein SQL-kyselyn, 10 johti tilanteeseen, jossa järjestelmä ei tuottanut lainkaan SQL-kyselyä, ja 3 tuotti virheellisen SQL-kyselyn. Tulokset osoittavat, että Cortex Analyst pystyy usein tuottamaan johdonmukaisia SQL-kyselyitä, mutta sen luotettavuus ei ollut kaikissa kysymystyypeissä riittävällä tasolla.

Herkkyystesteissä suurin variaatio havaittiin kysymyksen C eri muotoiluissa, joissa järjestelmä tuotti yhteensä kuusi uniikkia SQL-kyselyä. Kun Cortex Analystille

syötettiin kysymyksen C eri variaatioita, järjestelmä tuotti väärän vastauksen 14 kertaa 50 suorituksesta. Virheellisissä kyselyissä toistuvien ongelmien oli se, että järjestelmä ei rajannut hakua sarakkeen `direction` mukaisesti, vaan käytti saraketta `direction_original` tai jätti suuntasuodatuksen kokonaan pois.

Kysymyksessä C ja sen variaatioissa puhutaan palvelutasosta, jolloin implisiittisenä taustaoletuksena on, että tarkastelu kohdistuu vain saapuviin kontakteihin. Havainto viittaa siihen, että Cortex Analyst ei tässä tutkimusasetelmassa aina huomionnut tällaisia implisiittisiä oletuksia johdonmukaisesti. Luonnollisen kielen kysymyksissä esiintyvät alimääritellyt ilmaukset ja implisiittisen toimialatiedon päättely muodostavatkin tunnetun haasteen Text-to-SQL-malleille [10, 6].

Kun kysymys E ja sen variaatiot annettiin järjestelmälle syötteenä, järjestelmä tuotti 40 kertaa 50 suorituksesta tekstivastauksen, jossa se ilmoitti, että kysymykseen ei voida vastata. Variaatio E2 tuotti yhdeksän kertaa korrektein SQL-kyselyn ja kerran kyselyn, joka oli muuten oikea mutta käytti `distinct(identifier)`-ehdon sijaan ehtoa, jossa koko rivi vaadittiin uniikiksi. Tämä on huomattavasti heikompi ehto ja johti duplikaattivirheisiin.

Kysymyksen E variaatio E2 ei implikoi yhtä vahvasti, että kysymykseen vastaaminen vaatisi kahden rivin välistä vertailua. Tämä voi osaltaan selittää, miksi juuri tämä variaatio tuotti enemmän oikeita vastauksia kuin muut.

Kysymys E oli Cortex Analystille vaikein, mikä painottaa virhetyyppien jakaumaa ei kyselyä -luokan suuntaan.

Taulukosta 4 nähdään, että muista virhetyypeistä yleisimpiä olivat suodatusvirheet ja duplikaattien hallintaan liittyvät virheet. Kaikki suodatusvirheet liittyivät sarakkeisiin `direction` ja `direction_original`, mikä viittaa siihen, että järjestelmä voisi olla robustimpi, jos nämä kaksi saraketta erotettaisiin toisistaan semanttisesti selvemmin. Esimerkiksi `direction_original`-sarakkeen korvaaminen `is_callback`-sarakkeella saattaisi helpottaa mallia erottamaan sarakkeiden merkitykset toisistaan. Tätä ei kuitenkaan testattu tässä tutkimuksessa.

On kuitenkin huomioitava, että juuri nämä sarakkeet nousivat esiin tässä tutkimusasetelmassa. Toisenlaisessa tietokannassa tai toisessa sovellusympäristössä ongelmalliset kentät voisivat olla erilaisia. Havainto korostaa semanttisen näkymän merkitystä järjestelmän luotettavuuden kannalta. Semanttisen näkymän huolellinen suunnittelu, metadatan rikastaminen ja iteratiivinen kehittäminen voivat pitkällä aikavälillä parantaa järjestelmän toimintaa merkittävästi [38, 13].

Duplikaattien hallintavirheillä tarkoitetaan tässä virheitä, joissa `distinct`-avainsanaa on käytetty tarpeettomasti tai väärin. Tutkimusasetelmassa vain kysymystä A vastaavissa SQL-kyselyissä `distinct`-avainsanaa ei tarvita. Muiden kysymysten kohdalla duplikaattien hallinta tulisi yleensä kohdistaa `identifier`-sarakkeeseen. Duplikaattien hallintavirheitä esiintyi tutkimuksessa yhteensä 18 kertaa 250 suorituksen aikana.

Kysymyksessä A duplikaattien hallintaa ei odoteta lainkaan, mutta 10 suorituksessa järjestelmä tuotti SQL-kyselyn, joka sisälsi lausekkeen `distinct(identifier)`. Muut havaitut duplikaattien hallintavirheet liittyivät siihen, että `distinct`-avainsanaa sovellettiin n-tupleen, joka sisälsi `identifier`-sarakkeen lisäksi myös muita sarakkeita. Tällöin kysely vaatii näiden sarakkeiden yhdistelmän olevan uniikki, mikä on heikompi ehto kuin se, että `identifier`-sarake olisi jo yksin uniikki.

Taululiitos- tai sarakevalintavirheitä ei havaittu. Tämä viittaa siihen, että järjestelmä tunnisti kysymyksistä yleensä oikein, mitä attribuutteja käyttäjä halusi hakea, vaikka se ei aina onnistunut tuottamaan oikeaa SQL-kyselyä niiden perusteella.

Taulukko 2: Konsistenssikokeen tulokset.

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Erilaisten SQL- kyselyiden määrä
A	10	0	0	0	1
B	10	0	0	0	2
C	7	0	3	0	2
D	10	0	0	0	1
E	0	0	0	10	1

Taulukko 3: Herkkyystestien 1–5 tulokset. Sarakkeet ilmoittavat oikeiden, pienen virheen sisältävien, väärin ja ei kyselyä -vastausten lukumäärät sekä yhteenlaskettu uniikkien SQL-kyselyiden määrä. (*Yht.*).

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
A	10	0	0	0	1
A1	10	0	0	0	1
A2	10	0	0	0	1
A3	10	0	0	0	1
A4	0	10	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
B	10	0	0	0	2
B1	10	0	0	0	2
B2	10	0	0	0	2
B3	10	0	0	0	2
B4	10	0	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
C	7	0	3	0	2
C1	8	2	0	0	3
C2	10	0	0	0	3
C3	0	0	10	0	5
C4	8	1	1	0	6

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
D	10	0	0	0	1
D1	10	0	0	0	1
D2	10	0	0	0	1
D3	10	0	0	0	2
D4	7	0	3	0	3

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
E	0	0	0	10	1
E1	0	0	0	10	1
E2	9	1	0	0	3
E3	0	0	0	10	3
E4	0	0	0	10	3

Taulukko 4: Esiintyneiden virhetyyppien määrät kaikissa kokeissa. Rivit kuvaavat, kuinka monta kertaa kukin virhetyyppi esiintyi kussakin kysymyksessä tai sen variaatioissa. Jos yhdessä SQL-kyselyssä esiintyy useampi virhetyyppi, kysely lasketaan jokaiseen virhetyyppiin erikseen.

Kysymys	A	B	C	D	E	Yhteensä
Aggregaatio	0	0	2	0	0	2
Duplikaattien hallinta	10	0	7	0	1	18
Ei kyselyä	0	0	0	0	40	40
Sarakevalinta	0	0	0	0	0	0
Suodatus	0	0	11	3	0	14
Taululiitos	0	0	0	0	0	0

9 Rajoitukset

Tutkimukseen liittyy useita rajoituksia, jotka on huomioitava tuloksia tulkittaessa.

Ensinnäkin tutkimus kohdistuu yhteen järjestelmään, Snowflaken Cortex Analyysiin, jonka taustalla toimii Anthropicin Claude Sonnet 4 -malli. Tulokset kuvaavat siten yhtä tiettyä toteutusta eivätkä ole suoraan yleistettävissä muihin kielimalleihin, SQL-avustajiin tai luonnollisen kielen analytiikkatyökaluihin.

Toiseksi tutkimus perustuu yhteen sovellusalueeseen eli puhelinkeskusdataan. Tämän tietokannan rakenne, sanasto ja analyysitarpeet voivat poiketa merkittävästi muiden toimialojen vastaavista ympäristöistä. On mahdollista, että järjestelmä suoriutuisi eri tavoin esimerkiksi talousdatan, lokidatan tai asiakastietojen yhteydessä.

Kolmanneksi tutkimuksessa tarkastellaan mallin tuottamia SQL-kyselyitä eikä kyselyiden palauttamia varsinaisia vastauksia. Tämä rajaus on perusteltu aineiston luottamuksellisuuden vuoksi, mutta samalla se tarkoittaa, että tutkimus ei mittaa suoraan liiketoiminnallisen lopputuloksen oikeellisuutta. Kysely voi olla rakenteeltaan puutteellinen tai vaihtoehtoisesti syntaktisesti erilainen mutta käytännössä toimiva, eikä tätä eroa voida aina arvioida täysin ilman varsinaista tulosaineistoa.

Neljänneksi semanttisen näkymän laatu vaikuttaa suoraan siihen, miten hyvin Cortex Analyst kykenee muodostamaan kyselyitä. Jos näkymässä on epäselvyyksiä, puutteita tai monitulkintaisia kuvauksia, osa havaituista ongelmista voi johtua itse määrittelystä eikä pelkästään kielimallin rajoituksista. Tämän vuoksi tutkimus arvioi samalla myös sitä, kuinka hyvin kyseinen semanttinen näkymä tukee luonnollisen kielen ja tietokantarakenteen välistä tulkintaa.

Viidenneksi kielimallien toimintaa luonnehtii tietty epädeterministisyys, minkä vuoksi tulokset voivat vaihdella eri suorituskerroilla tai järjestelmäversioiden välillä. Lisäksi kaupalliset kielimallipohjaiset järjestelmät voivat muuttua ajan myötä ilman, että kaikki muutokset ovat käyttäjälle näkyviä. Tämän vuoksi tutkimuksen havainnot kuvaavat järjestelmän toimintaa tietyssä ajankohtana ja tietyssä konfiguraatiossa eivätkä välttämättä pysyvää ominaisuutta [8].

Kuudenneksi tuotettujen SQL-kyselyiden oikeellisuuden arvioi yksi arvioija. Tämä heikentää luokittelun reliabiliteettia ja lisää mahdollisuutta siihen, että osa luokitteluista sisältää arvioijan tulkintaan liittyvää subjektiivisuutta.

10 Johtopäätökset

Tämän tutkielman tavoitteena oli arvioida, kuinka hyvin Snowflaken Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin puhelinkeskusdataa sisältävässä tietokantaympäristössä. Tarkastelun kohteena olivat erityisesti järjestelmän kyky tuottaa oikeellisia SQL-kyselyitä, sen konsistenssi tilanteissa, joissa sama kysymys esitetään useita kertoja, sekä sen herkkyyssellaisille syötteen muotoilun muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä.

Tulokset osoittavat, että Cortex Analyst kykenee monissa tapauksissa tuottamaan oikeellisia ja keskenään johdonmukaisia SQL-kyselyitä. Erityisesti yksinkertaisemmat kysymykset sekä sellaiset kysymykset, joihin oli määriteltä valmiita metriikoita semanttisessa näkymässä, tuottivat hyvin vakaita tuloksia. Tämä viittaa siihen, että kielimallipohjainen luonnollisen kielen rajapinta voi toimia käyttökelpoisena välineenä tietokannan hyödyntämisessä silloin, kun kysymykset ovat rakenteeltaan selkeitä ja semanttinen näkymä tukee niitä riittävän hyvin.

Tutkimus osoitti kuitenkin myös merkittäviä rajoituksia. Järjestelmän suorituskyky heikkeni erityisesti silloin, kun kysymykseen sisältyi implisiittisiä taustaoletuksia tai kun oikean SQL-kyselyn muodostaminen edellytti tarkkaa suodatusta, duplikaattien hallintaa tai monitulkintaisen sanamuodon tulkintaa. Nämä havainnot ovat linjassa aiemman Text-to-SQL-tutkimuksen kanssa, jossa vastaavien monimutkaisten ehtojen ja taustatietojen yhdistämisen on todettu olevan kielimalleille huomattava haaste [17, 47]. Lisäksi havaittiin, että semanttisesti lähes saman sisältöiset kysymykset saattoivat johtaa erilaisiin SQL-kyselyihin ja joissakin tapauksissa myös erilaisiin virheisiin. Tämä ilmentää kielimalleille tyypillistä syöteherkkyyttä (*prompt sensitivity*), jossa pienetkin sanamuodon vaihtelut voivat muuttaa tuotettua lopputulosta merkittävästi [7], mikä heikentää luonnollisen kielen rajapinnan ennustettavuutta käyttäjän näkökulmasta.

Keskeinen havainto on, että järjestelmän luotettavuus ei riipu yksinomaan taustalla olevasta kielimallista, vaan myös siitä, miten hyvin semanttinen näkymä on suunniteltu. Tutkimuksessa havaitut virheet liittyivät usein tilanteisiin, joissa tietokannan kenttien merkitysten erottelu ei ollut mallin näkökulmasta riittävän selkeä. Tämän perusteella voidaan päätellä, että semanttinen näkymä ei ole pelkästään tekninen apurakenne, vaan keskeinen osa koko järjestelmän toimintaa. Sen huolellinen suunnittelu, yksiselitteinen nimeäminen ja iteratiivinen kehittäminen voivat parantaa luonnollisen kielen kyselyjärjestelmän luotettavuutta merkittävästi.

Tutkimuksen perusteella konsistenssia ei myöskään voida pitää yksinään riittävänä laadun mittarina. Järjestelmä saattoi tuottaa saman tuloksen toistuvasti myös silloin, kun tulos oli virheellinen tai kun SQL-kyselyä ei tuotettu lainkaan. Toisaalta joissakin tapauksissa samaan kysymykseen saatiin useita erilaisia SQL-kyselyitä, joista osa oli oikeellisia ja osa virheellisiä. Tämä osoittaa, että luonnollisen kielen tietokantarajapintojen arvioinnissa on tarkasteltava samanaikaisesti sekä oikeellisuutta että vaihtelua.

Kokonaisuutena tutkielma tukee alan yleistä näkemystä, jonka mukaan luonnollisen kielen käyttö tietokantojen rajapintana on lupaava mutta vielä osin epävarma lähestymistapa [19, 9]. Cortex Analyst voi toimia tehokkaana työkaluna erityisesti rajatuissa käyttötapauksissa, joissa tietokannan rakenne tunnetaan hyvin ja se

manttinen näkymä on laadittu huolellisesti. Sen sijaan tilanteissa, joissa kysymykset sisältävät implisiittisiä oletuksia, monitulkintaisuutta tai useita päättelyvaihteita, järjestelmän tuottamiin kyselyihin on syytä suhtautua varauksella.

Tutkielman tulokset viittaavat siihen, että käytännön kehitystyössä huomio kannattaa kohdistaa paitsi mallin suorituskykyyn myös siihen, miten järjestelmälle tarjottu semanttinen konteksti on rakennettu. Tulevassa tutkimuksessa olisi hyödyllistä vertailla useita erilaisia semanttisia näkymiä, testata järjestelmää muissa sovellusympäristöissä sekä tarkastella tuotettujen SQL-kyselyiden lisäksi myös niiden palauttamien vastausten oikeellisuutta. Lisäksi useamman arvioijan käyttö voisi parantaa luokittelun reliabiliteettia ja vahvistaa tulosten luotettavuutta.

Tämän tutkimuksen perusteella voidaan päätellä, että kielimallipohjaiset luonnollisen kielen tietokantarajapinnat eivät vielä täysin korvaa asiantuntijan laatimia kyselyitä, mutta ne voivat jo nyt toimia hyödyllisenä tukena analytiikkatyössä. Niiden käytännön arvo riippuu kuitenkin ratkaisevasti siitä, kuinka hyvin järjestelmän toimintaympäristö, semanttinen malli ja käyttötapa on sovitettu yhteen.

Viitteet

- [1] Anthropic. System card: Claude Sonnet 4. Technical report, Anthropic, March 2026.
- [2] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. Neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [3] Dimitri P. Bertsekas. *Nonlinear Programming*. Athena Scientific, 2 edition, 1999.
- [4] Léon Bottou. Online learning and stochastic approximations. *Online Learning and Neural Networks*, 1998. Revised, October 2010.
- [5] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.
- [6] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Xiang Deng, Kewei Tu, and Zhiguo Wang. Dr.Spider: A diagnostic evaluation benchmark towards robust text-to-SQL semantic parsing. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [7] Anwoy Chatterjee, H S V N S Kowndinya Renduchintala, Sumit Bhatia, and Tanmoy Chakraborty. POSIX: A prompt sensitivity index for large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14550–14565, 2024.
- [8] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT’s behavior changing over time? *Harvard Data Science Review*, (Special Issue 5), 2024.
- [9] Naihao Deng, Yulong Chen, and Yue Zhang. Recent advances in text-to-SQL: A survey of what we have and what we expect. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, pages 2166–2187, 2022.
- [10] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. Structure-grounded pretraining for text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1337–1350, 2021.
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.

- [12] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [13] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yuchen Zhu, Shi Chen, Zhi-Hong Deng, and Xing Zhang. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.
- [14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [15] Dan Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 3rd ed. draft edition, 2023.
- [16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015.
- [17] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, 2024.
- [18] Seppo Linnainmaa. The representation of the cumulative rounding error of an algorithm as a taylor expansion of the local rounding errors. *Master’s Thesis, University of Helsinki*, 1970.
- [19] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109*, 2024.
- [20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019.
- [21] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [22] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.
- [23] Yurii Nesterov. *Introductory Lectures on Convex Optimization*. Springer, 2004.
- [24] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2 edition, 2006.
- [25] Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

- [26] Mohammadreza Pourreza and Davood Rafiei. ETM: Modern insights into perspective on text-to-SQL evaluation in the age of large language models. *Future Internet*, 17(8):325, 2025.
- [27] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.
- [28] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *arXiv preprint arXiv:1904.09287*, 2018.
- [29] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [30] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [31] Frank Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, Washington, DC, 1962.
- [32] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [33] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2 edition, 2003.
- [34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- [35] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [36] Claude E. Shannon. Prediction and entropy of printed english. *Bell System Technical Journal*, 30(1):50–64, 1951.
- [37] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4596–4604. PMLR, 2018.
- [38] Snowflake Inc. Semantic model specification for Cortex Analyst, 2024. Viitattu: 24.8.2026.
- [39] Snowflake Inc. Cortex analyst. <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst>, 2026. Retrieved April 22, 2026.
- [40] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning*, pages 1139–1147. PMLR, 2013.

- [41] Tijmen Tieleman and Geoffrey Hinton. Lecture 6a - neural networks for machine learning. *COURSERA: Neural Networks for Machine Learning*, 2012. Lecture notes / Coursera course material.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [43] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.
- [44] Paul John Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, 1974.
- [45] Andrew G. Wilson and P. others. Marginal likelihood and the evidence lower bound (elbo). *arXiv preprint*, 2017.
- [46] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [47] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.